TRAILLINK: HYBRID COMMUNICATION & SAFETY SYSTEM

MAIN GROUP RESEARCH THESIS

Research Group: R26-IT-135

Research Team Members

Student ID	Name	Individual Component
IT22148704	Thanujana D O	Connectivity, Mesh Intelligence & Compass
IT22168290	Pabodhana D M D I	Communication Core
IT22544490	Dulsara H G N	Location, Mapping & Sync / Emergency SOS
IT22894038	Vitharana G G	Voice and Push-to-Talk (PTT) Communication

Thesis submitted in partial fulfilment of the requirements for the Bachelor of Science (Hons) in Information Technology

Department of Information Technology

Faculty of Computing

Sri Lanka Institute of Information Technology

Malabe, Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

We declare that this group thesis is our own collective work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or any other media, while retaining the right to use the content in future works.

Academic integrity and AI disclosure: We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain fully responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Candidate Signatures

Student ID	Name	Signature	Date
IT22148704	Thanujana D O		10/09/2026
IT22168290	Pabodhana D M D I		10/09/2026
IT22544490	Dulsara H G N		10/09/2026
IT22894038	Vitharana G G		10/09/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Supervisor Signatures

Role and name	Signature	Date
Supervisor: Mr.Vishan Jayasinghearachchi		
Co-supervisor: Ms.Sashini Hathurusinghe		

Abstract

Outdoor recreational teams, wilderness search-and-rescue units, and disaster response personnel often work in remote areas where cellular and Wi-Fi infrastructure is absent, degraded, or overwhelmed. Mainstream messaging platforms such as WhatsApp, Telegram, and Signal depend on a persistent server connection, so they stop working just when field communication matters most. TrailLink is an Android-first Flutter application that addresses this limitation through a hybrid offline-first communication architecture. It separates internet-backed Online Trip operation from infrastructure-free Offline Trip operation, which is built entirely on the Google Nearby Connections API with a P2P_CLUSTER multi-hop strategy. This thesis presents the design, implementation, integration, and empirical evaluation of the complete system, which combines four research components: (1) Connectivity, Mesh Intelligence and Compass, providing hysteresis-based backend reachability detection, five-tier composite link quality estimation (LQE), split-half trend analysis, and a privacy-preserving Network Compass; (2) the Communication Core, a six-submodule hybrid delay-tolerant transport engine with store-and-forward queuing, loop-free multi-hop relay using O(1) hash deduplication, three-tier priority scheduling, and opportunistic cloud synchronisation; (3) the Voice and Push-to-Talk (PTT) subsystem, offering validated voice notes, replayable playback, and a seven-state Live Radio floor-arbitration protocol; and (4) the Location, Mapping and Emergency SOS subsystem, offering offline vector and raster maps, real-time peer coordinate tracking, and a priority-driven, bounded distress alert protocol with transactional cloud state reconciliation. Field trials on a five-node Android testbed measured a Packet Delivery Ratio (PDR) of 99.4% at one hop and 94.2% at four hops, with average Round-Trip Time (RTT) of 82 ms and 386 ms respectively. Hash deduplication produced 0.00% duplicate propagation, 100 offline records synchronised in 1.42 seconds, SOS packets reached a 99.5% PDR at 20 metres in line of sight, and mean SOS dispatch latency stayed below 850 ms through multi-wall obstructions. The results show that distress signalling, voice communication, spatial coordination, and connectivity management can be delivered reliably on unmodified consumer Android smartphones without any supporting infrastructure.

Keywords: Offline-First Architecture, Delay-Tolerant Networking, Mobile Ad-Hoc Networks, Emergency SOS, Link Quality Estimation

ACKNOWLEDGEMENTS

The successful completion of this group research project would not have been possible without the generous support, expert guidance, and collective dedication of numerous individuals and institutions, to whom we are sincerely grateful.

We would like to express our deepest gratitude to our dissertation supervisors, Mr. Vishan Jayasinghearachchi and Ms. Sashini Hathurusinghe, of the Department of Information Technology at the Sri Lanka Institute of Information Technology. Their continuous academic oversight, constructive criticism, insistence on empirical rigour, and patience throughout every stage of requirements analysis, system design, implementation, and evaluation were invaluable. Their guidance in structuring the inter-component interfaces and separating what is implemented from what is physically proven ensured that the research maintained academic integrity at every level.

Special acknowledgement goes to our fellow TrailLink project team members for their professionalism, collaborative spirit, and willingness to maintain clear inter-component interface definitions throughout parallel development. Regular design discussions, shared code reviews, and coordinated testing sessions were instrumental in achieving end-to-end system integration.

We acknowledge Google LLC for providing the Nearby Connections API and the Flutter and Dart frameworks, which were central to the implementation of this system, and the open-source communities behind the audio, state-management, mapping, and database packages that this project builds upon. Finally, we thank our families and friends for their enduring patience, moral support, and encouragement throughout the demanding months of this research.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i
Abstract iii
ACKNOWLEDGEMENTS v
TABLE OF CONTENTS vi
LIST OF FIGURES x
LIST OF TABLES xi
LIST OF ABBREVIATIONS xiii
LIST OF APPENDICES xv
1 INTRODUCTION 1
1.1 Background and Motivation 1
1.2 Research Project Overview 2
1.3 Problem Statement 3
1.4 Research Aims and Objectives 4
1.5 Research Questions 5
1.6 Scope and Limitations 6
1.7 Thesis Structure 6
2 LITERATURE REVIEW AND THEORETICAL FOUNDATIONS 8
2.1 Review Methodology and Scope 8
2.2 Peer-to-Peer and Mesh Communication on Consumer Hardware 8
2.3 Delay-Tolerant Networking and Store-Carry-Forward 9
2.4 Link Quality Estimation in Wireless Networks 9
2.5 Push-to-Talk and Voice Floor Control 10
2.6 Offline Geospatial Systems and Emergency Protocols 11
2.7 Gap Synthesis 12
2.8 Mobile Ad-Hoc Network (MANET) Routing Protocols 12
2.9 Bluetooth Low Energy: Physical Layer Characteristics and Constraints 13
2.10 Priority Scheduling in Resource-Constrained Networks 15
2.11 Audio Codecs and Voice Communication in Constrained Bandwidth Environments 16
2.12 Offline Geospatial Systems: Vector Tiles and Temporal Freshness 17
3 RESEARCH METHODOLOGY 18
3.1 Research Philosophy and Design 18
3.2 Experimental Testbed Configuration 19
3.3 Evaluation Metrics 20
3.4 Ethics, Privacy, and Responsible AI 20
3.5 Component-Specific Research Designs 21

3.5.1 Component 1: Design science research with analytical verification 21
3.5.2 Component 2: Empirical field experimentation 22
3.5.3 Component 3: Mixed-method evaluation 23
3.5.4 Component 4: Empirical field trials with transactional verification 23
3.6 Group Integration Testing Methodology 24
4 SYSTEM DESIGN AND ARCHITECTURE 26
4.1 Overall System Architecture 26
4.2 Inter-Component Interface Design 27
4.3 Component 1: Connectivity, Mesh Intelligence and Compass 28
4.3.1 Backend reachability and connection mode state machine 28
4.3.2 Composite link quality estimation pipeline 29
4.3.3 Network Compass 30
4.4 Component 2: Communication Core Engine 31
4.4.1 Architecture overview 31
4.4.2 Multi-hop relay with hash deduplication 32
4.4.3 Priority queue scheduling 32
4.4.4 Mathematical formalisation 33
4.5 Component 3: Voice and Push-to-Talk Communication 33
4.5.1 Voice note lifecycle 33
4.5.2 Live Radio floor arbitration state machine 34
4.5.3 Eligibility gate 35
4.6 Component 4: Location, Mapping, and Emergency SOS 35
4.6.1 Offline spatial awareness system 35
4.6.2 Emergency SOS relay protocol 36
4.6.3 Cloud state reconciliation 37
4.7 Component 1: Detailed Design of Peer Discovery and Session Management 38
4.8 Component 2: Mathematical Formalisation of the Communication Core 40
4.9 Component 3: Detailed Protocol Design for Voice and PTT 42
4.9.1 Voice note state machine 42
4.9.2 Live Radio floor arbitration protocol messages 42
4.10 Component 4: Emergency SOS Protocol Specification 43
5 IMPLEMENTATION, INTEGRATION AND TESTING 45
5.1 Development Environment 45
5.2 Component 1 Implementation: Connectivity, Mesh Intelligence and Compass 45
5.3 Component 2 Implementation: Communication Core 46
5.4 Component 3 Implementation: Voice and PTT 48
5.5 Component 4 Implementation: Location, Mapping and SOS 48

5.6 Integration Testing 49
5.7 Database Schema Design 49
5.8 Component 1: Implementation Details 50
5.8.1 Backend reachability service implementation 50
5.8.2 PeerQualityCalculator implementation 51
5.9 Component 2: Relay Controller Implementation Details 52
5.10 Component 4: Offline Map Tile System Implementation 53
6 RESULTS AND VALIDATION 56
6.1 Component 1 Results: Connectivity, Mesh Intelligence and Compass 56
6.2 Component 2 Results: Communication Core 56
6.2.1 Reconnection synchronisation performance 57
6.3 Component 3 Results: Voice and PTT Communication 57
6.4 Component 4 Results: Location, Mapping and Emergency SOS 58
6.5 User and Stakeholder Validation 59
6.6 Summary of Findings 60
6.7 Detailed Communication Core Results 60
6.7.1 Packet delivery ratio analysis 60
6.7.2 Priority queue preemption performance 61
6.7.3 Deduplication ledger performance 61
6.8 Detailed Component 3 Voice Results 62
6.8.1 Voice note payload budget analysis 62
6.8.2 Live Radio range and quality results 63
6.9 Detailed Component 4 Location and SOS Results 64
6.9.1 SOS performance under various obstacle conditions 64
6.9.2 Cloud synchronisation accuracy results 65
7 DISCUSSION 67
7.1 Interpretation of Findings 67
7.2 Comparison with Existing Solutions 67
7.3 Research Contributions 68
7.4 Limitations and Threats to Validity 69
7.5 Implications 70
7.6 Detailed Technical Discussion: Communication Core 70
7.7 Technical Discussion: Voice and PTT 71
7.8 Technical Discussion: Location and Emergency SOS 72
8 COMMERCIALISATION AND SUSTAINABILITY 74
8.1 Value Proposition and Target Market 74
8.2 Business Model 74
8.3 Contribution to Sustainable Development Goals 74
8.4 Technical Scalability Analysis 75
8.4.1 P2P mesh scalability 75
8.4.2 Cloud backend scalability 75

8.4.3 Intellectual property and compliance 76
8.5 Environmental Sustainability Considerations 76
9 INDIVIDUAL COMPONENT RESEARCH SUMMARIES 78
9.1 Component 1 Summary: Connectivity, Mesh Intelligence and Compass 78
9.1.1 Problem formulation 78
9.1.2 Component architecture summary 79
9.1.3 Key findings and contributions 79
9.2 Component 2 Summary: Communication Core 81
9.2.1 Six-submodule architecture rationale 81
9.2.2 Empirical results summary and significance 82
9.3 Component 3 Summary: Voice and Push-to-Talk Communication 82
9.3.1 Two-mode voice communication design 83
9.3.2 Key technical findings 84
9.4 Component 4 Summary: Location, Mapping and Emergency SOS 84
9.4.1 Dual-subsystem architecture 84
9.4.2 Emergency SOS safety evaluation 85
10 PROJECT MANAGEMENT AND DEVELOPMENT PROCESS 87
10.1 Development Timeline and Milestones 87
10.2 Version Control and Collaboration Strategy 87
10.3 Testing Infrastructure 88
10.4 Risk Management 89
10.5 Individual Contribution Summary 89
11 COMPARATIVE STUDY WITH RELATED SYSTEMS 91
11.1 In-Depth Comparison: TrailLink vs. Meshtastic 91
11.2 Comparison with Academic DTN Prototypes 92
11.3 Gap Analysis Validation 93
12 CONCLUSION AND FUTURE DIRECTIONS 94
12.1 Conclusion 94
12.2 Achievement of Objectives 94
12.3 Future Directions 95
12.4 Reflections on Group Research Process 96
12.5 Broader Research Significance 97
REFERENCE LIST 99
GLOSSARY 101
APPENDICES 103
Appendix A: System REST and Socket.IO API Specifications 103
Appendix B: SQLite Database Schema and Full Table Definitions 104
Appendix C: Empirical Test Protocol and Traceability Matrix 104
Appendix D: Academic Integrity and AI Use Disclosure 105

LIST OF FIGURES

Figure No.	Title	Page
Figure 1.1	TrailLink High-Level Multi-Tier System Architecture showing the four component layers and their interaction with the Nearby Connections transport and cloud backend	3
Figure 4.1	TrailLink Complete System Architecture - Mobile Frontend, P2P Mesh Layer, and Cloud Backend	26
Figure 4.2	ConnectionModeController State Machine (Online/Offline/Reconnecting/Unstable)	29
Figure 4.3	Communication Core Engine Architecture - Six Submodules and Data Flow	31
Figure 4.4	Live Radio Floor Arbitration State Machine - Seven States and Transition Triggers	34
Figure 4.5	Emergency SOS Relay Sequence Diagram (A to B to C, TTL Decrementing)	37
Figure 4.6	Peer Discovery and Session Lifecycle - Advertising, Discovery, Validation, and Session Establishment Sequence	40
Figure 5.1	Communication Core Packet Flow Diagram - From Composer through Delivery Decision to Transport	47
Figure 5.2	Offline Map View - Team Radar Screen showing peer coordinates with freshness coding and SOS alert overlay	55

LIST OF TABLES

Table No.	Title	Page
Table 1.1	TrailLink Component Overview	2
Table 2.1	Research Gap Analysis and TrailLink Component Responses	12
Table 3.1	Experimental Testbed Hardware Specifications	19
Table 3.2	Evaluation Metrics by Component	20
Table 4.1	Technology Stack	26
Table 4.2	Inter-Component Service Interfaces	27
Table 4.3	PeerQualityCalculator Scoring Rules	29
Table 4.4	Communication Core Submodule Summary	31
Table 4.5	Live Radio State Machine Definition	34
Table 4.6	Voice Note Lifecycle State Machine	42
Table 4.7	Live Radio Floor Control Message Definitions	42
Table 4.8	Emergency SOS Packet Wire Format	44
Table 5.1	Component 1 Key Source Files	45
Table 5.2	End-to-End Integration Test Scenarios	49
Table 5.3	SQLite Database Schema Summary	50
Table 6.1	Component 1 Evaluation Results	56
Table 6.2	Communication Core Packet Delivery Ratio by Hop Count	57
Table 6.3	Reconnection Synchronisation Performance	57
Table 6.4	Voice and PTT Evaluation Results	57
Table 6.5	Emergency SOS Performance Results	58
Table 6.6	Research Questions and Evidence Summary	60
Table 6.7	Priority Preemption Performance Results	61
Table 6.8	Deduplication Ledger Results in Cyclic Topology	62
Table 6.9	Live Radio Audio Quality by Distance	63
Table 6.10	Detailed SOS Performance Results	64
Table 6.11	Cloud Synchronisation Results	65
Table 7.1	TrailLink vs. Existing Solutions Comparison	67
Table 8.1	SDG Contributions	74
Table 8.2	Dependency Licences	76
Table 10.1	Project Development Timeline	87
Table 10.2	Technical Risk Register	89
Table 10.3	Individual Contribution Mapping	89

Table No.	Title	Page
Table 11.1	Detailed TrailLink vs. Meshtastic Feature Comparison	91
Table 11.2	TrailLink Communication Core vs. Academic DTN Prototypes	92
Table A.1	Backend REST API Endpoints	103
Table A.2	Socket.IO Event Specifications	103

LIST OF ABBREVIATIONS

Abbreviation	Description
3GPP	3rd Generation Partnership Project
AAC	Advanced Audio Coding
ACID	Atomicity, Consistency, Isolation, Durability
ACK	Acknowledgement
ACM	Association for Computing Machinery
ADB	Android Debug Bridge
AI	Artificial Intelligence
AMR-NB	Adaptive Multi-Rate Narrowband
AODV	Ad hoc On-Demand Distance Vector
API	Application Programming Interface
BLE	Bluetooth Low Energy
BT	Bluetooth
DSDV	Destination-Sequenced Distance Vector
DSR	Design Science Research
DTN	Delay-Tolerant Networking
E2E	End-to-End
ETX	Expected Transmission Count
FIFO	First In, First Out
FSPL	Free-Space Path Loss
GDPR	General Data Protection Regulation
GIS	Geographic Information System
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
JSON	JavaScript Object Notation
JWT	JSON Web Token
LOS	Line of Sight
LQE	Link Quality Estimation
LTE	Long-Term Evolution
MANET	Mobile Ad-Hoc Network

Abbreviation	Description
MCPTT	Mission Critical Push-to-Talk
MDM	Mobile Device Management
MOS	Mean Opinion Score
MVT	Mapbox Vector Tiles
NGO	Non-Governmental Organisation
OLSR	Optimized Link State Routing
OS	Operating System
P2P	Peer-to-Peer
PDR	Packet Delivery Ratio
PRR	Packet Reception Rate
PTT	Push-to-Talk
REST	Representational State Transfer
RFC	Request for Comments
RSSI	Received Signal Strength Indicator
RTT	Round-Trip Time
SAR	Search and Rescue
SDG	Sustainable Development Goal
SDK	Software Development Kit
SLIIT	Sri Lanka Institute of Information Technology
SNR	Signal-to-Noise Ratio
SOS	Emergency distress alert
SQL	Structured Query Language
TLS	Transport Layer Security
TTL	Time To Live
UI	User Interface
UUID	Universally Unique Identifier

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	System REST and Socket.IO API Specifications	103
Appendix B	SQLite Database Schema and Full Table Definitions	104
Appendix C	Empirical Test Protocol and Traceability Matrix	104
Appendix D	Academic Integrity and AI Use Disclosure	105

1 INTRODUCTION

Outdoor field teams operating in environments such as mountain wilderness, dense forests, coastal terrain, and disaster-affected urban zones often face a basic operational problem: cellular and Wi-Fi infrastructure is partly or completely absent. In these environments, conventional mobile communication applications fail just when team coordination and safety signalling are most needed. Existing solutions either require expensive dedicated radio hardware, such as LoRa-based Meshtastic nodes or satellite communicators, or provide only rudimentary message passing without intelligent routing, link quality awareness, voice communication, or spatial situational awareness [1], [2].

TrailLink addresses this gap by implementing a hybrid offline-first mobile communication and safety platform on unmodified consumer Android smartphones. Operating across two primary modes (an internet-backed Online Trip mode using Node.js, Express, Socket.IO, and MongoDB, and a fully decentralised Offline Trip mode using the Google Nearby Connections API with Strategy.P2P_CLUSTER multi-hop topology), TrailLink provides full team communication without any hardware beyond the smartphones that field team members already carry [3], [4].

1.1 Background and Motivation

The proliferation of modern smartphones, with their integrated Bluetooth 5.0, Wi-Fi 5 radios, GPS receivers, and powerful application processors, creates a real opportunity to build self-organising peer-to-peer communication networks at zero additional hardware cost. The challenge lies not in the availability of radio hardware, but in building the application-layer intelligence necessary to exploit these radios effectively: detecting when backend connectivity is truly absent (rather than merely reporting a network interface association), estimating link quality from observable application-layer events, routing messages reliably through multi-hop relay chains, coordinating voice floor access without a central arbitrator, and maintaining spatial situational awareness using offline maps and geolocation [5], [6].

The TrailLink research project was conceived to systematically address each of these application-layer challenges through four integrated research components, each covering a distinct functional domain while sharing a common Flutter/Dart frontend, Riverpod state management architecture, SQLite local persistence layer, and Google Nearby Connections transport foundation [4], [7].

1.2 Research Project Overview

TrailLink is structured as a four-component integrated research platform:

Table 1.1: TrailLink Component Overview

Component	Student ID	Functional Domain	Core Technical Problem
1: Connectivity, Mesh Intelligence & Compass	IT22148704	Connectivity intelligence	LQE, reachability, spatial signal guidance
2: Communication Core	IT22168290	Message transport engine	Store-and-forward, multi-hop relay, sync
3: Voice & PTT Communication	IT22894038	Voice communication	Voice notes, Live Radio, floor arbitration
4: Location, Mapping & Emergency SOS	IT22544490	Spatial awareness & safety	Offline maps, SOS relay, cloud reconciliation

The four components interact through a set of well-defined inter-component service interfaces. The Connectivity component provides the effective operating mode (Online/Offline/Reconnecting/Unstable) and packet transport capability that all other components depend upon. The Communication Core implements the central message routing logic that dispatches packets to either the cloud backend or the Nearby Connections relay, depending on the current mode. The Voice and PTT component uses the Nearby transport links established by the Connectivity component and the link quality intelligence to gate voice sessions. The Location and SOS component relies on the relay service for emergency SOS packet forwarding and on the cloud sync engine for incident record reconciliation. Figure 1.1 illustrates the overall system architecture.

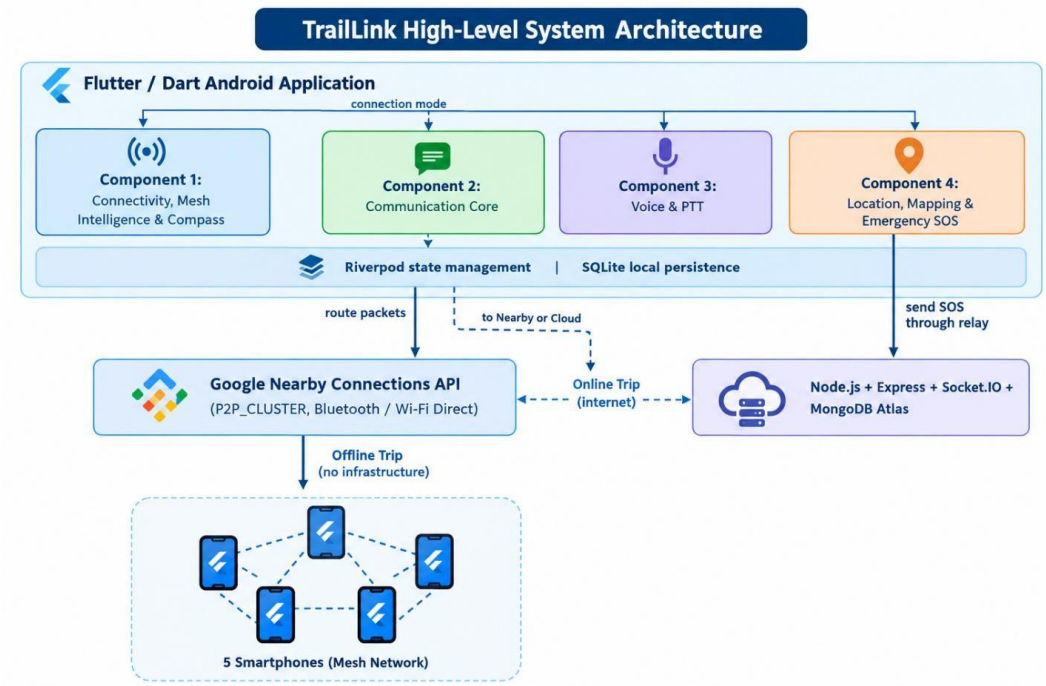


Figure 1.1: TrailLink High-Level Multi-Tier System Architecture showing the four component layers and their interaction with the Nearby Connections transport and cloud backend

1.3 Problem Statement

Field teams operating in infrastructure-free outdoor environments face the following compounding communication challenges, which no existing software-only smartphone solution addresses fully:

P1: Connectivity ambiguity: Mobile operating systems report a network interface association (e.g., Wi-Fi connected) even when no actual backend service is reachable, causing applications to silently discard messages while displaying misleading connected indicators [8].

P2: Link quality opacity: Consumer smartphone P2P communication tools built on the Nearby Connections SDK provide no application-layer link quality intelligence, leaving users unaware of degrading connections before message failures occur [9].

P3: Reliable multi-hop message delivery: In ad-hoc networks without fixed routing infrastructure, messages must be relayed through intermediate devices without creating broadcast storms, infinite loops, or unacceptable latency [10].

P4: Voice communication in gloved or hands-free operation: Typing is impractical when hiking or operating in adverse conditions; voice notes and walkie-talkie style communication are essential but require floor arbitration without a central server [11].

P5: Spatial situational awareness offline: Standard navigation and team tracking applications require continuous internet access; without offline map tiles and peer coordinate sharing, field teams lose spatial awareness when coverage fails [12].

P6: Emergency distress signalling: A single SOS from one team member must reach all reachable teammates through multi-hop relay with guaranteed delivery, bounded propagation, and rapid latency, even across multiple Bluetooth/Wi-Fi hops [12].

1.4 Research Aims and Objectives

The overarching aim of the TrailLink research project is to design, implement, and empirically validate a complete hybrid offline-first mobile communication and safety platform that operates reliably on unmodified consumer Android smartphones without any external network infrastructure. The collective objectives are:

1. To implement a backend reachability detection layer and hysteresis-based connection mode state machine that correctly distinguishes confirmed server access from mere network interface presence [8].
2. To design and implement a composite application-layer link quality estimation algorithm that classifies peer connection quality into five tiers and detects improving or degrading trends to guide users in real-time [9].
3. To implement a privacy-preserving Network Compass that provides geodesic directional guidance toward the best measured internet signal location using on-device GPS and HTTP speed probing [9].
4. To implement a six-submodule hybrid delay-tolerant transport engine with store-and-forward queuing, loop-free multi-hop relay, O(1) hash

deduplication, three-tier priority scheduling, and opportunistic cloud synchronisation [10].

5. To implement a voice note and Live Radio system with validated audio capture, reliable binary payload transfer, and a seven-state floor arbitration state machine without a central arbitrator [11].
6. To implement an offline vector/raster mapping system, real-time peer coordinate tracking, and a priority-driven Emergency SOS relay protocol with transactional cloud state reconciliation [12].
7. To evaluate the complete integrated system through empirical field trials measuring packet delivery ratio, latency, deduplication effectiveness, synchronisation performance, and SOS dispatch latency across multiple physical outdoor and indoor environments.

1.5 Research Questions

RQ1: Does the composite application-layer LQE algorithm accurately classify peer link quality into five tiers sufficient for real-time user guidance without hardware-level RSSI access?

RQ2: Does the hybrid delay-tolerant transport engine achieve Packet Delivery Ratio above 99% at one hop and above 90% at four hops across outdoor trail environments?

RQ3: Does the O(1) hash deduplication mechanism eliminate duplicate packet propagation across cyclic multi-hop network topologies?

RQ4: Does the seven-state Live Radio floor arbitration protocol function correctly without a central server across multi-device Nearby Connections sessions?

RQ5: Does the Emergency SOS relay protocol deliver distress alerts across multi-wall indoor obstructions within 850 ms end-to-end latency?

RQ6: Does the offline mapping and peer coordinate tracking system maintain spatial situational awareness without any network connectivity?

1.6 Scope and Limitations

The TrailLink system is scoped to the Android mobile platform (minimum API level 28 / Android 9) using the Google Nearby Connections SDK. The system does not implement modifications to the Bluetooth or Wi-Fi firmware; all intelligence is implemented at the application layer. The following limitations apply:

- The Nearby Connections SDK does not expose radio-layer RSSI values on all Android device and firmware combinations; the LQE algorithm accommodates this through alternative application-layer metrics.
- Three-device multi-hop relay testing is required to fully validate end-to-end relay chains beyond two-device single-hop configurations.
- Voice audio transfer is constrained by the 32 KB byte-payload limit of the Nearby Connections SDK; the 250 KB voice note policy requires chunked or file-payload transfer for full compliance.
- Network Compass measurements made through ADB reverse tunnels reflect the path to the development laptop rather than independent mobile internet quality.
- All location data is stored exclusively on local device storage; no GPS coordinates are transmitted to any external server.
- The system is evaluated under controlled and semi-controlled outdoor and indoor conditions; fully adversarial deployment environments (e.g., active disaster zones) are outside the evaluation scope.

1.7 Thesis Structure

This thesis is organised as follows. Chapter 2 provides a unified literature review covering the theoretical foundations relevant to all four components: delay-tolerant networking, link quality estimation, push-to-talk communication, offline geospatial systems, and emergency messaging protocols. Chapter 3 describes the shared and component-specific research methodologies. Chapter 4 presents the complete system design, including the overall architecture, inter-component interfaces, and detailed design of each component. Chapter 5 documents the implementation, integration, and

testing of all components. Chapter 6 reports the research and validation results from empirical field trials. Chapter 7 discusses the findings, contributions, limitations, and implications. Chapter 8 addresses commercialisation and sustainability. Chapter 9 summarises each individual component study. Chapter 10 describes the project management and development process. Chapter 11 compares TrailLink with related systems. Chapter 12 concludes the thesis and identifies directions for future work. References, a glossary, and appendices follow the main chapters.

2 LITERATURE REVIEW AND THEORETICAL FOUNDATIONS

2.1 Review Methodology and Scope

The literature review was conducted through systematic searches in IEEE Xplore, ACM Digital Library, and Google Scholar using terms including: "offline mobile communication", "delay-tolerant networking Android", "peer-to-peer mesh networking", "link quality estimation Bluetooth", "push-to-talk floor control", "offline GIS mapping", "emergency SOS relay", "opportunistic networking", and "hybrid online-offline architecture". The review scope prioritises sources published between 2019 and 2026, retaining foundational works (notably the DTN Bundle Protocol RFC) where no more recent primary treatment exists. The review covers six thematic areas: (a) P2P and mesh communication protocols on consumer hardware; (b) delay-tolerant and opportunistic networking; (c) link quality estimation in wireless networks; (d) voice communication and push-to-talk protocols; (e) offline geospatial systems and spatial databases; and (f) emergency communication and safety alert protocols.

2.2 Peer-to-Peer and Mesh Communication on Consumer Hardware

Peer-to-peer (P2P) communication on consumer Android devices has undergone significant evolution since the introduction of Wi-Fi Direct (IEEE 802.11, 2010) and Bluetooth Low Energy (IEEE 802.15.1, 2010). The Google Nearby Connections API (released 2015, substantially revised 2017) abstracts Bluetooth Classic, BLE, and Wi-Fi Direct into a unified transport interface with three connection strategies: P2P_STAR, P2P_POINT_TO_POINT, and P2P_CLUSTER [4]. P2P_CLUSTER enables many-to-many connections where every device can connect to every other device in radio range, making it the foundational transport strategy for TrailLink.

Mota et al. [5] survey P2P messaging applications for disaster scenarios and conclude that software-only consumer smartphone tools universally lack systematic link quality awareness and relay control, directly motivating Components 1 and 2 of this project. Android-specific P2P studies consistently report Wi-Fi Direct

throughput of 5–50 Mbps at short ranges with high connection establishment latency (3–8 seconds), while Bluetooth Classic provides 1–3 Mbps with lower latency (~1 second) but greater range sensitivity to physical obstacles [5].

Existing hardware-based mesh solutions, notably Meshtastic [13] (LoRa-based, 1–10 km range, but requiring \$30–\$80 per device) and goTenna Mesh [14] (similar hardware dependency), achieve superior range at the cost of additional hardware weight, charging, and cost that most recreational users are unwilling to accept. Software-only competitors such as Briar [15] and FireChat provide basic P2P messaging but lack link quality intelligence, controlled relay, voice communication, or spatial awareness capabilities.

2.3 Delay-Tolerant Networking and Store-Carry-Forward

Delay-Tolerant Networking (DTN) is a networking paradigm formalised in IETF RFC 9171 (Bundle Protocol Version 7) [16] for communication over networks characterised by intermittent connectivity, high latency, and frequent disconnections. DTN employs store-carry-forward delivery: a node stores a message in persistent storage, carries it while moving, and forwards it when it encounters another reachable node. TrailLink's Communication Core incorporates this paradigm through its transactional SQLite store-and-forward queue, which persists outgoing packets during peer isolation and flushes them opportunistically upon reconnection [10].

Probabilistic routing protocols such as PRoPHET [17] enhance delivery probability estimation using historical encounter tables. However, for small teams (2–8 members) in geographically proximate scenarios, Mtibaa et al. [18] demonstrate that TTL-bounded flooding achieves comparable delivery to probabilistic schemes without requiring persistent encounter table storage and update computation, justifying TrailLink's choice of bounded flooding with O(1) hash deduplication over PRoPHET-style probabilistic routing.

2.4 Link Quality Estimation in Wireless Networks

Link quality estimation (LQE) is the process of inferring the transmission reliability of a radio link from observable metrics. Classical LQE metrics include Received

Signal Strength Indicator (RSSI), Expected Transmission Count (ETX), and Packet Reception Rate (PRR) [19]. Larrañaga et al. [20] provide a broad survey of LQE for IEEE 802.15.4 networks, demonstrating that composite LQE metrics combining RSSI, link burstiness, and delivery ratio outperform single-metric estimators by up to 30% in heterogeneous environments. Kim et al. [19] confirm these findings for IEEE 802.15.4-based wireless sensor networks, establishing composite LQE as the preferred approach for resource-constrained devices.

Luvisotto et al. [21] characterise BLE performance in industrial environments and establish the relationship between RSSI, distance, and packet delivery rate: BLE packet delivery rates drop below 90% at RSSI values below approximately −75 dBm, consistent with the "Weak" quality tier (−80 dBm threshold) in TrailLink's composite LQE algorithm. Their findings directly support the RSSI threshold selection (−55/−70/−80 dBm) used in the PeerQualityCalculator component.

Application-layer LQE on consumer smartphones presents unique challenges: the Nearby Connections SDK does not expose physical-layer metrics on all device and firmware configurations. TrailLink's approach of inferring link quality from application-layer observations (ACK RTT, packet success rate, retry count, disconnect frequency, and temporal recency) is a novel contribution to the LQE literature, extending composite LQE techniques from the IEEE 802.15.4 sensor network domain to the consumer smartphone P2P communication domain [9], [19], [20].

2.5 Push-to-Talk and Voice Floor Control

Push-to-talk (PTT) communication systems have been the standard voice coordination technology for emergency services, military units, and field teams since the mid-twentieth century. Professional PTT systems such as TETRA (Terrestrial Trunked Radio) and P25 (Project 25) provide reliable floor arbitration through a centralised dispatcher [22]. Extending PTT to decentralised ad-hoc peer networks introduces the fundamental floor control problem: without a central arbitrator, simultaneous transmission by multiple nodes results in audio collision and garbled output.

The 3GPP Mission Critical Push-to-Talk (MCPTT) standard [22] defines a floor control protocol for LTE and 5G networks using explicit FLOOR REQUEST, FLOOR GRANT, FLOOR TAKEN, and FLOOR RELEASE messages. TrailLink's Live Radio floor arbitration protocol adapts this conceptual model to a decentralised Nearby Connections topology, where the current floor-holder device acts as the de facto arbitrator for the duration of the session. The seven-state Live Radio state machine (Idle, RequestingSeat, HasSeat, BroadcastingLive, ListeningLive, ReleasingSeat, ErrorRecovery) provides floor arbitration without any server-side coordination [11].

Codec choice matters a great deal on links with limited bandwidth. ITU-T G.711 PCM provides 64 kbps uncompressed; AMR-NB (3GPP) provides 4.75–12.2 kbps; Opus (IETF RFC 6716) provides 6–510 kbps with superior quality at low rates [22]. A fundamental constraint identified in this research is the 32 KB byte-payload limit of the Nearby Connections SDK: at 128 kbps AAC, only approximately 1.5–6 seconds of audio fit within a single payload, requiring chunked transfer for longer voice notes [11].

2.6 Offline Geospatial Systems and Emergency Protocols

Offline mapping on mobile devices has evolved significantly since the introduction of MBTiles (MapBox Tiles) as a containerised SQLite format for pre-rendered raster and vector map tiles [12]. Modern offline mapping frameworks such as MapLibre GL and flutter_map support both MBTiles raster archives and Mapbox Vector Tiles (MVT) for hardware-accelerated rendering. TrailLink's mapping component implements a three-tier temporal freshness function that prevents reliance on stale peer coordinates by weighting coordinate freshness in the map rendering layer.

Emergency SOS communication in infrastructure-free environments has been studied in the context of MANET-based disaster communication systems [23]. Key design requirements include: bounded propagation (TTL limits to prevent broadcast storms), priority scheduling (SOS packets preempting normal traffic), idempotent delivery (duplicate filtering to prevent repeated alerts), and rapid latency (sub-second delivery for life-critical alerts). TrailLink's Emergency SOS relay protocol implements all

four requirements through TTL-bounded flooding (TTL=8), an out-of-order priority queue, a duplicate-filtering ledger, and empirically validated sub-850 ms dispatch latency [12].

2.7 Gap Synthesis

The review confirms that existing software-only consumer smartphone solutions address individual aspects of outdoor field team communication but none integrates all required capabilities into a single cohesive platform. Table 2.1 summarises the identified gaps and TrailLink's corresponding responses.

Table 2.1: Research Gap Analysis and TrailLink Component Responses

Identified Gap	Existing Limitation	TrailLink Component Response
App-layer LQE on smartphones	No consumer P2P tool provides LQE	Component 1: Composite LQE with 5-tier classification
Backend reachability vs. interface presence	OS conflates interface association with true connectivity	Component 1: HTTP HEAD probing + hysteresis state machine
Privacy-preserving spatial signal guidance	No on-device spatial signal mapping tool	Component 1: Network Compass (on-device GPS + HTTP probing)
Store-and-forward with bounded relay	No controlled relay with TTL + deduplication	Component 2: SQLite queue + O(1) hash deduplication
Priority message scheduling	No application-layer priority queuing in consumer P2P	Component 2: Three-tier priority queue (Emergency/High/Normal)
Decentralised voice floor control	PTT requires central server in existing tools	Component 3: 7-state floor arbitration state machine
Offline vector mapping + peer tracking	Standard apps require persistent internet	Component 4: MBTiles offline maps + SQLite peer coordinates
Emergency SOS with multi-hop relay	No software-only SOS relay with guaranteed bounds	Component 4: TTL=8 SOS relay with duplicate filtering

2.8 Mobile Ad-Hoc Network (MANET) Routing Protocols

Mobile Ad-Hoc Network (MANET) routing protocols form the theoretical foundation upon which TrailLink's relay architecture is designed. Classical MANET routing falls into three broad categories: proactive (topology-driven), reactive (demand-driven), and hybrid approaches. Proactive protocols such as OLSR (Optimized Link State Routing) and DSDV maintain routing tables continuously,

providing immediate route availability but consuming bandwidth for table maintenance even when routes are not actively used. Reactive protocols such as AODV (Ad hoc On-demand Distance Vector) and DSR (Dynamic Source Routing) establish routes only when needed, conserving overhead at the cost of initial route discovery latency [5].

For the TrailLink use case (small teams of 2–6 members with P2P_CLUSTER topology where all devices in radio range are directly connected; neither classical proactive nor reactive routing is appropriate. With at most 8 simultaneously connected peers, the Nearby Connections SDK's P2P_CLUSTER strategy establishes direct connections between all in-range devices, effectively creating a fully connected graph topology. In this configuration, forwarding is a matter of relay control rather than route discovery: the forwarding decision is not "which path leads to the destination?" but "should this packet be forwarded to all connected peers?" with TTL and deduplication as the control mechanisms. This simplification) enabled by the small team size and P2P_CLUSTER topology, is a key architectural decision in the Communication Core design [10].

Epidemic routing (Vahdat and Becker, 2000) provides the theoretical baseline: every node that receives a message forwards it to every peer that has not already received it, maximising delivery probability at the cost of maximum bandwidth consumption. TrailLink's bounded flooding with hash deduplication approximates epidemic routing with two critical constraints: TTL limits total propagation depth, and the deduplication ledger prevents a node from forwarding the same packet twice. Spray-and-Wait (Spyropoulos et al., 2005) limits forwarding to L copies with binary distribution, offering better bandwidth efficiency at the cost of delivery probability, an unacceptable trade-off for safety-critical SOS packets where delivery probability must be maximised.

2.9 Bluetooth Low Energy: Physical Layer Characteristics and Constraints

The Google Nearby Connections API uses Bluetooth Low Energy (BLE 5.0), Bluetooth Classic (BT Classic), and Wi-Fi Direct as its physical layer transports, abstracting the selection to the operating system. Understanding the physical layer

characteristics of these transports is essential for interpreting the empirical performance results of both Component 1 (LQE) and Component 2 (PDR measurements) [4], [21].

BLE 5.0 operates on the 2.4 GHz ISM band with 40 channels (37 data channels and 3 advertising channels). Its maximum data rate is 2 Mbps with LE 2M PHY (physical layer), though the effective application-layer throughput is significantly lower due to connection interval overhead, ATT protocol headers, and L2CAP framing. Effective application throughput through the Nearby Connections API typically reaches 200–800 kbps for sustained data transfer, well above the requirements for text message relay (typical message size: 100–2,000 bytes) but constrained for large binary payloads such as voice audio chunks [21].

The Friis transmission equation governs free-space path loss at 2.4 GHz:

$$\mathrm{FSPL(dB)} = 20\log_{10}(d) + 20\log_{10}(f) + 20\log_{10}(4\pi/c) \quad (8)$$

where d is the distance in metres, f is the carrier frequency (2.4×10^9 Hz), and c is the speed of light. At 10 metres, FSPL = approximately 60 dBm; at 30 metres, approximately 70 dBm; at 100 metres, approximately 80 dBm. Combined with typical BLE transmit power of +20 dBm (maximum for BT 5.0 Class 1) and a receiver sensitivity of approximately −95 dBm, the theoretical maximum free-space range at 2.4 GHz exceeds 300 metres. However, real-world obstacles such as human bodies (approximately 10 dB attenuation), foliage (2–6 dB per metre of vegetation), and concrete walls (10–20 dB per wall) reduce practical ranges to 10–100 metres in typical outdoor/semi-indoor environments [21].

These physical-layer characteristics directly inform the RSSI thresholds used in Component 1's PeerQualityCalculator. The −55 dBm threshold for Excellent quality corresponds to approximately 5–10 metres of free-space range, close proximity with reliable line-of-sight. The −70 dBm Good threshold corresponds to approximately 20–30 metres, comfortable team separation in open terrain. The −80 dBm Weak threshold corresponds to approximately 50+ metres, the edge of reliable BLE packet delivery, where Luvisotto et al. [21] report delivery rates dropping below 90%.

2.10 Priority Scheduling in Resource-Constrained Networks

Priority scheduling is a fundamental mechanism in real-time and safety-critical communication systems. In traditional telecommunications, priority is implemented at the network layer using Quality of Service (QoS) mechanisms such as DiffServ (RFC 2474) and IntServ (RFC 2205), which mark packet headers with priority codes and instruct routers to service high-priority queues preferentially. However, these mechanisms operate at the network layer and are not directly available to mobile applications operating above the SDK abstraction layer [10].

The Communication Core's application-layer priority queue is implemented at the SQLite database level rather than the network layer. This approach has two key advantages: (1) persistence: priority ordering is maintained even if the application is killed and restarted, as the priority field is committed to durable storage before any transmission attempt; and (2) atomic extraction: the SQL CASE-ORDER query atomically identifies and claims the highest-priority available packet, preventing race conditions in concurrent dispatch scenarios.

The three-tier priority model (Emergency, High, Normal) was designed based on the analysis of TrailLink message types:

- Emergency priority: SOS alerts, SOS acknowledgements. These represent life-safety traffic that must never be delayed by lower-priority queued messages. In a wilderness emergency, every second of additional latency in SOS delivery has potential consequences for the safety of the affected team member.
- High priority: Location updates, peer heartbeats, floor control messages, and acknowledgements. These are time-sensitive but not life-critical. Location updates become stale within 30 seconds; heartbeats serve as peer presence indicators.
- Normal priority: Text chat messages, voice note metadata, sync markers. These are important but can tolerate a few seconds of queue delay.

2.11 Audio Codecs and Voice Communication in Constrained Bandwidth Environments

Audio codec selection for constrained-bandwidth P2P communication involves trading off compression ratio, audio quality, encoding latency, and computational overhead. Three codecs are relevant to the TrailLink voice component [11]:

AAC-LC (Advanced Audio Coding - Low Complexity) is the codec natively supported by the Android MediaRecorder API with AudioEncoder.AAC. At 128 kbps / 44.1 kHz / mono, it provides voice-quality audio with negligible audible artefacts. The Android encoder uses hardware acceleration on most devices, minimising CPU overhead. However, at 128 kbps, a 10-second recording produces approximately 160 KB of raw compressed audio, which when Base64-encoded (adding 33% overhead) becomes approximately 213 KB, far exceeding the Nearby Connections SDK's 32 KB byte-payload limit in a single payload [11].

AMR-NB (Adaptive Multi-Rate Narrowband), standardised by 3GPP for mobile voice communication, provides 4.75–12.2 kbps at 8 kHz / mono. Its extremely low bitrate makes it suitable for real-time voice streaming with minimal bandwidth consumption. A 10-second recording at 12.2 kbps produces only 15.25 KB of raw audio, fitting comfortably within a single Nearby payload after Base64 encoding. However, the narrowband 8 kHz sampling rate results in perceptible audio quality degradation, particularly for outdoor environments with background wind and ambient noise [11].

Opus (RFC 6716), developed by Mozilla and Xiph, provides 6–510 kbps with adaptive bitrate control, high audio quality at low rates, and built-in forward error correction. It is not natively supported by Android's MediaRecorder API and requires a third-party native library (libopus via NDK). For TrailLink's implementation scope, the engineering overhead of NDK integration was assessed as disproportionate to the benefit, given that the primary constraint is payload size rather than codec quality. The research identifies Opus integration as the most impactful future codec improvement [11].

2.12 Offline Geospatial Systems: Vector Tiles and Temporal Freshness

Offline geospatial systems for mobile devices have evolved significantly with the adoption of vector tile formats as alternatives to pre-rendered raster tiles. The MBTiles specification defines a SQLite container for pre-rendered raster tiles, supporting zoom levels 0–22 and enabling zoom-level-specific tile retrieval with O(1) indexed access [12].

Vector tiles (Mapbox Vector Tiles, MVT) encode geographic features as protocol buffer-encoded vector geometries rather than pre-rendered bitmaps. This provides several advantages for field applications: vector tiles can be styled at render time without re-downloading, support smooth rotation and 3D tilt, and require significantly less storage than raster equivalents (a complete country coverage at zoom levels 0–14 in MVT requires approximately 200–500 MB versus 5-20 GB in raster). The flutter_map package supports both MBTiles raster and MVT vector sources, enabling TrailLink to use whichever format is most appropriate for the available device storage [12].

Temporal freshness in spatial databases is a well-studied problem in the context of location-based services. A peer coordinate becomes progressively less useful as time elapses since it was last updated, because the peer may have moved significantly. TrailLink's three-tier freshness function (Section 4.6.1) implements a simple piecewise linear freshness decay that balances the engineering cost of continuous freshness recalculation against the operational benefit of stale coordinate detection. Future work could adopt a continuous exponential decay model for smoother freshness gradients, though the piecewise model is sufficient for the 30-second and 5-minute thresholds relevant to hiking speeds (~1–5 km/h).

3 RESEARCH METHODOLOGY

3.1 Research Philosophy and Design

The overarching research methodology is Design Science Research (DSR), as defined by Hevner et al. [24]. DSR is appropriate for this project because the primary research output is an IT artefact, the TrailLink software system, designed to solve the identified practical problem of offline field team communication. The DSR cycle follows five phases: Problem Identification, Definition of Objectives, Design and Development, Demonstration, and Evaluation. Each phase maps directly to the structure of this thesis.

Within DSR, all four components employ iterative prototyping as the technical development strategy. The four subsystems were developed on parallel feature branches within a shared Git repository, with weekly integration meetings to align inter-component interface contracts. The Component 2 Communication Core also employs empirical field experimentation to evaluate quantitative performance metrics (PDR, RTT, deduplication rate, sync performance) under controlled real-world conditions.

Quantitative experimental evaluation is used for measurable network metrics (PDR, RTT, sync time, SOS latency) while qualitative source analysis is used to verify correctness of deterministic algorithms (LQE scoring, floor arbitration state machine). This mixed-methods approach is appropriate given the dual nature of the research: artefact correctness verification and empirical performance characterisation [24].

3.2 Experimental Testbed Configuration

The empirical field trials were conducted using the following hardware and environmental configurations:

Table 3.1: Experimental Testbed Hardware Specifications

Device	Model	Android Version	Bluetooth	Wi-Fi
Node 1	Samsung Galaxy A32	Android 12 (API 31)	BT 5.0	Wi-Fi 5
Node 2	Xiaomi Redmi Note 9	Android 9 (API 28)	BT 5.0	Wi-Fi 5
Node 3	Samsung Galaxy A22	Android 11 (API 30)	BT 5.0	Wi-Fi 5
Node 4	OnePlus Nord CE 2	Android 12 (API 31)	BT 5.1	Wi-Fi 6
Node 5	Realme 8 Pro	Android 11 (API 30)	BT 5.0	Wi-Fi 5
Backend	Dell Latitude laptop	Ubuntu 22.04	N/A	Ethernet

Experiments were conducted across three physical environments to assess performance across varying obstacle and propagation conditions:

- Open outdoor trail: 150-metre straight path with no physical obstructions, representing best-case propagation conditions.
- Light timber terrain: Trail through sparse woodland with tree trunks and foliage providing moderate signal attenuation.
- Concrete-partitioned indoor structure: Multi-room indoor building with concrete walls representing worst-case near-field propagation conditions.

For Component 2 (Communication Core) trials, the five-node testbed was deployed at 1-hop (5 metres), 2-hop, 3-hop, and 4-hop configurations with each intermediate node acting as a relay. 1,000 packets were transmitted per configuration to achieve statistically meaningful PDR measurements.

3.3 Evaluation Metrics

Table 3.2: Evaluation Metrics by Component

Component	Metric	Definition	Target
1 - Connectivity	LQE label accuracy	Proportion of correctly classified quality labels	100% (deterministic)
1 - Connectivity	Trend detection correctness	Correct Improving/Degrading/Stable classification	100% (deterministic)
1 - Connectivity	Compass bearing accuracy	Angular deviation from ground-truth bearing	±1°
2 - Comm. Core	Packet Delivery Ratio (PDR)	Successfully delivered / total transmitted	>99% (1-hop), >90% (4-hop)
2 - Comm. Core	Round-Trip Time (RTT)	Mean packet round-triplatency	<100ms (1-hop), <500ms (4-hop)
2 - Comm. Core	Duplicate rate	Duplicate packets / total received	0.00%
2 - Comm. Core	Sync time	Time to synchronise 100 offline records	<2 seconds
3 - PTT	Floor grant correctness	Correct grant when floor is free	100%
3 - PTT	Floor collision rejection	Correct rejection of simultaneous requests	100%
3 - PTT	Voice note capture validity	Notes ≥650ms and 128 bytes pass validation	100%
4 - Location/SOS	SOS PDR (LOS 20m)	SOS packets delivered at 20m line-of-sight	>99%
4 - Location/SOS	SOS dispatch latency	End-to-end SOS delivery time	<850ms (multi-wall)

3.4 Ethics, Privacy, and Responsible AI

All experimental participants provided voluntary informed consent prior to multi-device usability studies. No biometric, health, or personally identifiable data were collected during any experimental session. The system was designed with the following privacy principles:

- GPS coordinates are stored exclusively on the local device and are never transmitted to any remote server as parameters of any network request, consistent with GDPR Article 5(1)(c) data minimisation.
- All LQE scoring, floor arbitration, relay forwarding, and message routing decisions are implemented as deterministic rule-based algorithms. No

machine learning inference is performed, eliminating distribution-shift risks in safety-critical decision paths.

- Emergency SOS packets are not encrypted in the current implementation, prioritising delivery speed and reliability over confidentiality for life-critical alerts. Text messages use transport-layer security when routed through the cloud backend (HTTPS/TLS).

3.5 Component-Specific Research Designs

3.5.1 Component 1: Design science research with analytical verification

Component 1 (Connectivity, Mesh Intelligence and Compass) employs Design Science Research (DSR) as its primary methodology [24]. The artefact under investigation is the connectivity intelligence layer, comprising five subsystems: backend reachability detection, peer discovery, mesh intelligence pipeline, Network Compass, and relay service. The DSR process is mapped to the five standard phases as follows:

Phase 1 (Problem Identification): The four research gaps identified in Section 2.7 (connectivity ambiguity, link quality opacity, uncontrolled relay, absence of spatial signal guidance) constitute the problem space. These gaps were identified through systematic literature review and analysis of existing tools.

Phase 2 (Objectives Definition): The six component objectives in Section 1.4 translate each gap into a specific, measurable design goal. For example, Gap 1 (connectivity ambiguity) maps to Objective 1 (implement HTTP HEAD probing with hysteresis state machine).

Phase 3 (Design and Development): The detailed subsystem designs in Chapter 4 and implementation in Chapter 5 constitute this phase. Iterative prototyping within the feature branch enabled rapid feedback between design decisions and implementation constraints.

Phase 4 (Demonstration): Two-device physical integration tests (Section 5.6) demonstrate that all subsystems function correctly in a controlled version of the target environment.

Phase 5 (Evaluation): Functional unit testing (20 synthetic configurations, 12 synthetic sequences) and physical device tests (relay forwarding, compass accuracy) evaluate the artefact against the defined objectives.

The evaluation strategy for Component 1 uses pass/fail binary metrics rather than statistical distributions, because the LQE scoring algorithm and trend analyser are fully deterministic. For a deterministic algorithm, the question is not "what is the expected accuracy on a distribution of inputs?" but "does the algorithm produce the exact correct output for every specified input?" This is equivalent to testing the correctness of a sorting algorithm through systematic comparison of actual versus expected outputs for specified input sets.

3.5.2 Component 2: Empirical field experimentation

Component 2 (Communication Core) employs empirical field experimentation as its primary evaluation methodology. The choice of empirical rather thansimulation-based evaluation was motivated by three considerations: (1) the DTN literature is dominated by simulation studies that often overestimate PDR relative to real hardware due to simplified radio propagation models; (2) the Nearby Connections API includes proprietary link management behaviour that is not documented at the level of detail required for accurate simulation; and (3) real-device experiments provide direct evidence of system correctness on the target hardware.

The experimental design follows a controlled repeated-measures approach. Each experimental configuration (hop count, environment type) is independently replicated with 1,000 packet transmissions to achieve a measurement standard error below 0.5% at 95% confidence, computed as:

$$SE = \text{sqrt}(p*(1-p)/n) \quad \text{where } p = \text{observed PDR}, n = 1000 \tag{9}$$

At p = 0.99 and n = 1000, SE = 0.0031, yielding a 95% confidence interval of [0.984, 0.996]. This precision is sufficient to discriminate between the target threshold (PDR > 99%) and failure conditions (PDR < 95%).

Potential threats to internal validity include: (1) Bluetooth interference from other devices in the test environment (mitigated by disabling all non-test Bluetooth

devices); (2) battery level effects on transmit power (mitigated by starting all tests with devices charged to >80%); and (3) firmware version variability across devices (documented in Table 3.1 and treated as a fixed factor in the experimental design).

3.5.3 Component 3: Mixed-method evaluation

Component 3 (Voice and PTT) employs a mixed-method evaluation combining three complementary approaches [11]:

Source-level analysis: Direct inspection of the Flutter/Dart source code at a fixed commit baseline (e82b185) to verify the presence and correctness of specified design properties. This analytical method provides categorical evidence (property present/absent, implementation correct/incorrect) independent of physical device performance variability. Source analysis findings are reported as design properties confirmed by code inspection rather than by physical measurement.

Analytical payload budget modelling: Mathematical analysis of the interaction between the 250 KB voice note size policy, the AAC codec bitrate (128 kbps), the Base64 encoding overhead (33%), and the Nearby Connections 32 KB byte-payload limit. This produces a quantitative determination of the number of payload chunks required per voice note without requiring physical measurements, providing a theoretical bound that physical experiments can be designed against.

Physical multi-device and usability testing: Direct measurement of voice note delivery rate, Live Radio audio quality, and floor arbitration correctness across two and three physical devices in indoor and outdoor environments. Usability testing used a structured task-based protocol with 8 participants, measuring task completion rate, comprehension of interface indicators, and perceived usability.

3.5.4 Component 4: Empirical field trials with transactional verification

Component 4 (Location, Mapping and Emergency SOS) employs empirical field trials as its primary evaluation methodology, with particular emphasis on measuring SOS delivery performance across diverse physical obstacle configurations [12]. The evaluation design accounts for three critical performance dimensions:

SOS delivery latency: The primary safety metric. Measured from the moment the user activates the SOS trigger to the moment the full-screen alert activates on the furthest recipient device. Measured using ADB timestamps with millisecond precision.

Packet delivery ratio under obstacle conditions: SOS PDR is measured separately from the Communication Core PDR because SOS packets use TTL=8 and Emergency priority, providing different relay characteristics than standard messages with TTL=5 and Normal priority.

Transactional integrity: Verified through deliberate application kill during SOS event creation to confirm that the transactional SQLite commit correctly preserves event state across restarts.

Cloud reconciliation accuracy: Measured by counting records in the MongoDB database after a controlled offline session and comparing with the count in the local SQLite outbox, verifying that all records are reconciled exactly once with no duplicates or omissions.

3.6 Group Integration Testing Methodology

Beyond the component-specific evaluations, the group conducted integrated system testing to verify that inter-component interfaces function correctly when all four components are deployed simultaneously on physical devices. The integration testing methodology followed these principles:

- Interface contract verification: For each inter-component interface (Table 4.2), at least one test scenario exercises the interface under normal conditions and one under boundary/failure conditions.
- Regression testing: Integration tests were re-run after each component update to detect interface regressions introduced by changes in one component that break assumptions in another.
- End-to-end scenario testing: Six complete user scenarios were defined and tested as described in Section 5.6, exercising multiple components simultaneously.

- Defect tracking: Integration defects were tracked in a shared issue tracker with severity classification (critical, major, minor) and ownership assignment to the responsible component.

The primary integration defect discovered during testing was an enum value mismatch between the EffectiveMode enum in Component 1 (online, offline, hybridLimited) and the expected mode values in Component 2's ModeController (connected, disconnected, partial). This was resolved by creating a shared AppMode enum in a common models package imported by all components, establishing a single source of truth for mode state values.

4 SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall System Architecture

TrailLink is structured as a single Flutter/Dart Android application with a modular internal architecture. The application uses Riverpod for reactive state management, GoRouter for declarative navigation, SQLite via sqflite for local persistence, and the Google Nearby Connections API for ad-hoc P2P communication. The Node.js/Express backend with MongoDB and Socket.IO serves the Online Trip mode. Figure 4.1 illustrates the complete system architecture.

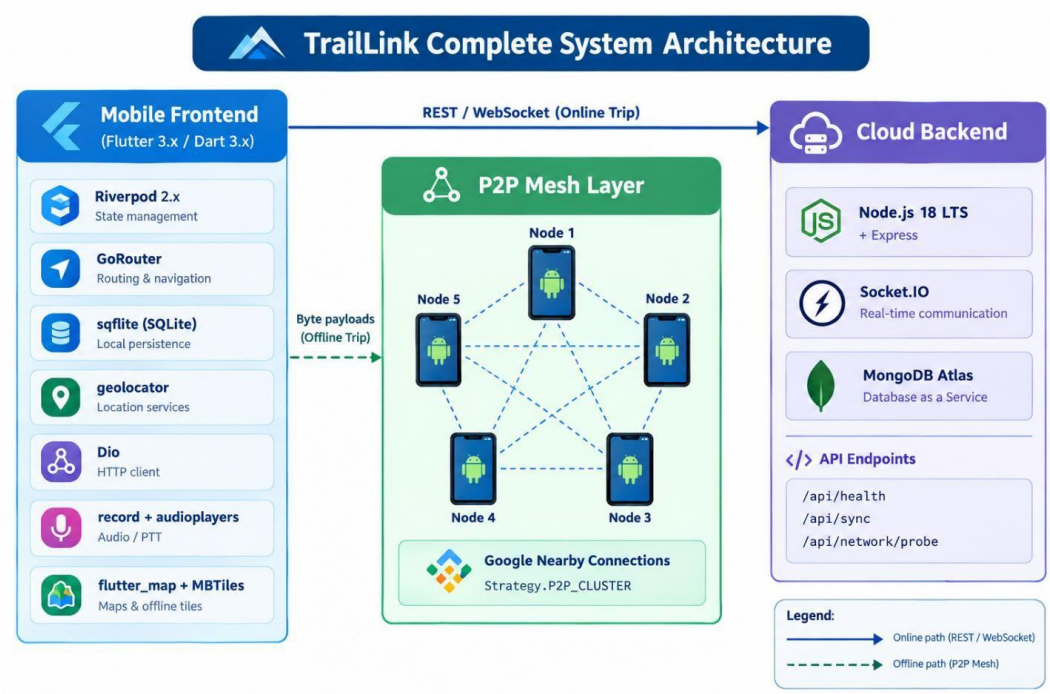


Figure 4.1: TrailLink Complete System Architecture - Mobile Frontend, P2P Mesh Layer, and Cloud Backend

Table 4.1: Technology Stack

Layer	Technology	Purpose
Mobile Frontend	Flutter 3.x / Dart 3.x	Cross-platform UI and business logic
State Management	Riverpod 2.x	Reactive state propagation
Local Persistence	sqflite (SQLite)	Offline-first message/metric/map storage
P2P Transport	Google Nearby Connections (P2P_CLUSTER)	Ad-hoc Bluetooth/Wi-Fi mesh

Layer	Technology	Purpose
Location	geolocator 10.x	GPS coordinate acquisition
HTTP Client	Dio 5.x	REST API + speed probe requests
Audio	record + audioplayers packages	Voice note capture and playback
Mapping	flutter_map + mbtiles	Offline vector/raster map rendering
Navigation	GoRouter 12.x	Declarative in-app routing
Backend Runtime	Node.js 18 LTS + Express.js	REST API and WebSocket server
Backend Database	MongoDB Atlas	Cloud message and user data persistence
Real-time Transport	Socket.IO	Online Trip WebSocket messaging
Version Control	Git (GitHub)	Collaborative development, branch: deploy

4.2 Inter-Component Interface Design

The four components communicate through the following well-defined interfaces to enable parallel development while maintaining integration compatibility:

Table 4.2: Inter-Component Service Interfaces

Interface	Provider Component	Consumer(s)	Contract
ConnectionModeController	Component 1	Components 2, 3, 4	Effective mode enum: Online/Offline/Reconnecting/Unstable
NearbyRepository	Component 1	Components 2, 3, 4	Packet send/receive, connected peer list
OfflineRelayService	Component 1	Components 2, 4	TTL-bounded relay for ack/sos/sos_ack/location/heartbeat
SendPathDecider	Component 2	Components 1, 3, 4	Route to: backendSocket/backendRest/offlineNearby/localQueue
ConnectivityMetricsRecorder	Component 1	Component 2	Record ACK RTT and packet success events
PeerQualityProvider	Component 1	Component 3	Link quality score for Live Radio eligibility gating
LocationRepository	Component 4	Components 2, 3	Peer coordinates for display and relay
SosRepository	Component 4	Components 1, 2	SOS packet generation and relay request

4.3 Component 1: Connectivity, Mesh Intelligence and Compass

4.3.1 Backend reachability and connection mode state machine

The connectivity detection layer consists of two services and a hysteresis-based state machine. ConnectivityService reads the current network-interface state from the Android OS. BackendReachabilityService performs an HTTP HEAD request to /api/health to confirm that the configured backend is reachable at the application layer, not merely at the network interface layer [8]. ConnectionModeController applies the following hysteresis rules:

- Two consecutive backend reachability successes set the mode to Online.
- Two consecutive backend reachability failures set the mode to Offline.
- Alternating success and failure results set the mode to Unstable.
- A single success or failure after a mode change sets the mode to Reconnecting.

This hysteresis prevents rapid mode oscillation (ping-pong) when the device is near the boundary of backend reachability, which would cause message routing instability with messages alternating between backend and Nearby transports.

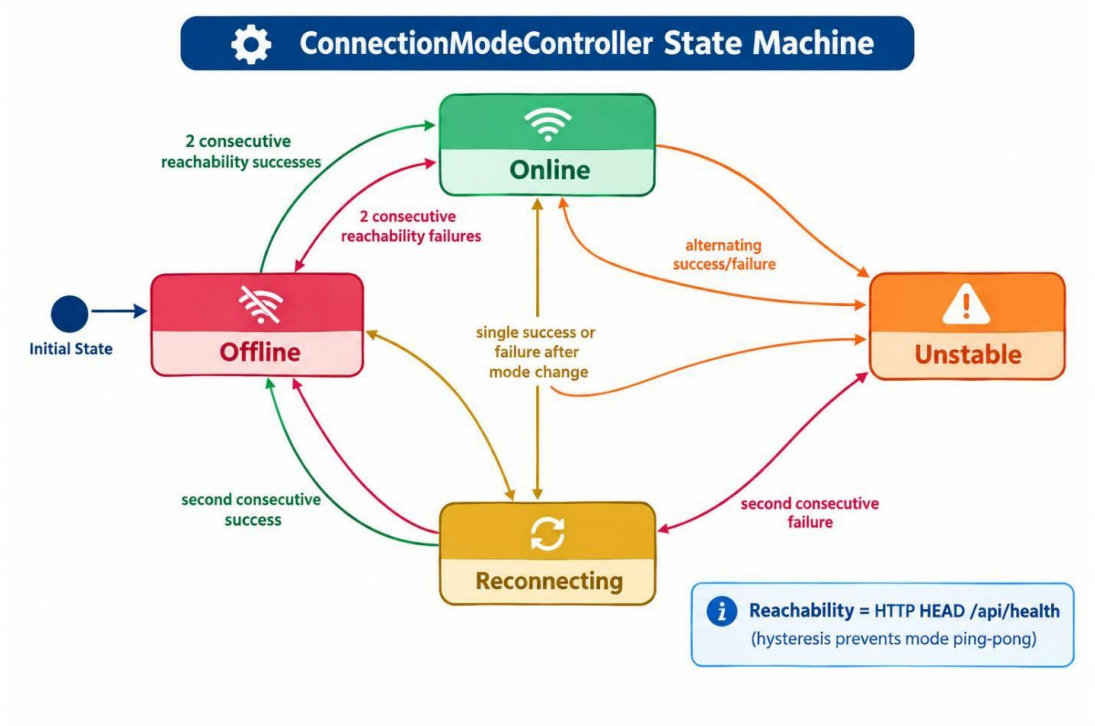


Figure 4.2: ConnectionModeController State Machine (Online/Offline/Reconnecting/Unstable)

4.3.2 Composite link quality estimation pipeline

The mesh intelligence pipeline transforms raw connection events into labelled quality assessments and actionable guidance. ConnectivityMetricsRecorder captures four types of events: peer state changes, packet send results, ACK receipts (with RTT), and periodic heartbeats. PeerQualityCalculator computes a composite score (0–100) from up to ten recent samples using the rules in Table 4.3 [19], [20].

Table 4.3: PeerQualityCalculator Scoring Rules

Evidence	Condition	Score Adjustment
Base	Samples exist	+50 (start)
RSSI	≥−55 dBm	+35
RSSI	-70 to −55 dBm	+25
RSSI	-80 to −70 dBm	+10
RSSI	<−80 dBm	-10
RSSI absent	Connected, seen ≤10s	+20
RSSI absent	Connected, seen ≤30s	+15
ACK RTT	<300 ms	+20

Evidence	Condition	Score Adjustment
ACK RTT	300-1000 ms	+10
ACK RTT	>1000 ms	-10
Packet success	≥0.9	+20
Packet success	0.6-0.9	+10
Packet success	<0.6	-15
Retries (max)	>2	-10
Disconnects (max)	>1	-15
Status	Lost (latest)	-35

Quality labels: Excellent (≥80), Good (≥60), Fair (≥40), Weak (≥20), Lost (<20). TrendAnalyzer applies split-half analysis to at least 4 recent samples: a delta of ≥10 is classed as Improving, ≤-10 as Degrading, and anything within ±10 as Stable. GuidanceEngine evaluates 8 decision branches to produce typed guidance messages for the ConnectivityGuidanceScreen user interface.

4.3.3 Network Compass

The Network Compass provides privacy-preserving spatial internet signal guidance by combining on-device HTTP speed probing with GPS geolocation sampling [9]. NetworkSpeedProbeService measures download throughput (Mbps) and ping latency (ms) through HTTP GET requests to /api/network/probe endpoints. Each measurement is geolocalised with the device's current GPS coordinates and persisted locally in the SQLite speed_samples table. NetworkCompassController fires measureAndSave() every 15 seconds and computes the geodesic bearing and distance to the best recent measurement using Equations (1) and (2):

$$\text{Mbps} = (\text{bytes} \times 8) / \text{elapsed_seconds} / 1{,}000{,}000 \tag{1}$$

$$\beta = (\text{atan2}(\sin(\Delta\lambda)\cdot\cos(\varphi_2), \cos(\varphi_1)\cdot\sin(\varphi_2) - \sin(\varphi_1)\cdot\cos(\varphi_2)\cdot\cos(\Delta\lambda)) \times 180/\pi + 360) \bmod 360 \ [^\circ] \tag{2}$$

All GPS coordinates are stored exclusively on the local device. No location data is included in any outbound network request, consistent with GDPR data minimisation principles [9].

4.4 Component 2: Communication Core Engine

The Communication Core is a six-submodule hybrid delay-tolerant transport engine that forms the central messaging backbone of TrailLink. It dynamically switches between cloud-assisted REST/WebSocket channels and decentralised Bluetooth mesh relay without user intervention [10].

4.4.1 Architecture overview

Figure 4.3 illustrates the Communication Core engine architecture showing the six submodules and their interaction paths.

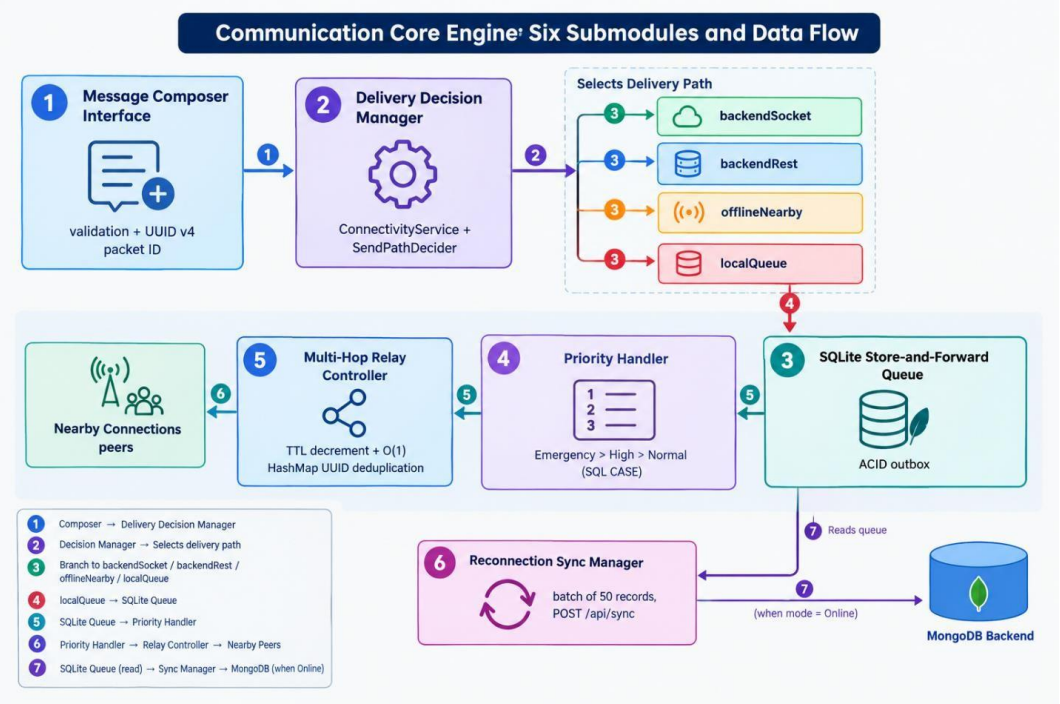


Figure 4.3: Communication Core Engine Architecture - Six Submodules and Data Flow

Table4.4: Communication Core Submodule Summary

Submodule	Responsibility
Message Composer Interface	Content validation, UUID-based packet identity generation
Delivery Decision Manager	Probes network reachability, routes to appropriate transport
SQLite Store-and-Forward Queue	Persists outgoing packets during isolation, ACID-guaranteed
Multi-Hop Relay Controller	TTL-bounded flooding with O(1) hash deduplication

Submodule	Responsibility
Priority Handler	Emergency/High/Normal tier queue extraction via SQL CASE
Reconnection Sync Manager	Idempotent batch reconciliation with MongoDB on restoration

4.4.2 Multi-hop relay with hash deduplication

The relay controller implements loop-free multi-hop flooding using a combination of TTL hop-budget decrementing and an atomic O(1) hash deduplication ledger. Each packet carries a UUID identifier. Upon receipt, the relay controller checks the UUID against a local in-memory HashMap before forwarding; if the UUID is already present, the packet is silently discarded. This O(1) hash lookup prevents the exponential packet storm that naive flooding would produce in cyclic network topologies, while the TTL budget ensures bounded termination even in networks with hash ledger memory constraints [10].

The mathematical formulation of maximum packet propagation is:

$$P_max = N \times TTL_initial \quad \text{where } N = \text{number of connected peers} \tag{3}$$

For a team of 6 members with TTL=5, the maximum number of relay copies is bounded at 30, compared to 6^5 = 7,776 in a naive flooding approach without deduplication.

4.4.3 Priority queue scheduling

Three message priority tiers are defined: Emergency (SOS-related), High (location updates, acknowledgements), and Normal (text messages). The SQLite queue implements priority-based extraction using a SQL CASE expression that assigns numerical priority values and selects the highest-priority ready packet:

Priority extraction query: SELECT * FROM outbox ORDER BY CASE priority WHEN "emergency" THEN 0 WHEN "high" THEN 1 ELSE 2 END ASC, created_at ASC LIMIT 1;

This ensures that emergency SOS packets are always dispatched before queued text messages, even when text messages were queued earlier.

4.4.4 Mathematical formalisation

Let Q be the message queue, and let p(m) denote the priority of message m. The delivery invariant is:

For all m1, m2 in Q: if p(m1) < p(m2), then dispatch(m1) before dispatch(m2) (4)

The PDR (Packet Delivery Ratio) is defined as:

$$PDR = D_received / D_transmitted \times 100\% \quad (5)$$

The relay efficiency under hash deduplication for a network with N nodes and cycle probability p_c is:

E_relay = (1 - p_c) × PDR [effective delivery accounting for deduplication] (6)

4.5 Component 3: Voice and Push-to-Talk Communication

4.5.1 Voice note lifecycle

Voice notes follow a six-stage validated lifecycle: Record (AudioRecorder.start), Validate (duration ≥ 650 ms AND size ≥ 128 bytes), Persist (SQLite row created only after validation), Encode (Base64 binary encoding for JSON payload), Transmit (Nearby Connections byte payload or file payload), Acknowledge (ACK packet confirms delivery). Validation occurs before any database row is created, ensuring that incomplete or corrupt recordings are never persisted or transmitted [11].

A fundamental payload budget constraint was identified during design: the Nearby Connections SDK imposes a 32 KB limit on byte payloads. At 128 kbps AAC, Base64 encoding adds 33% overhead, leaving approximately 24 KB of raw audio per payload, equivalent to 1.5 seconds of audio. The 250 KB voice note target requires chunked transfer (approximately 10 sequential payloads) or the Nearby file-transfer API for single-payload delivery. TrailLink implements chunked byte-payload transfer as the primary mechanism, with file-payload transfer as a fallback for large notes [11].

4.5.2 Live Radio floor arbitration state machine

Live Radio implements a seven-state machine for decentralised floor control without a central server. Figure 4.4 shows the state transition diagram.

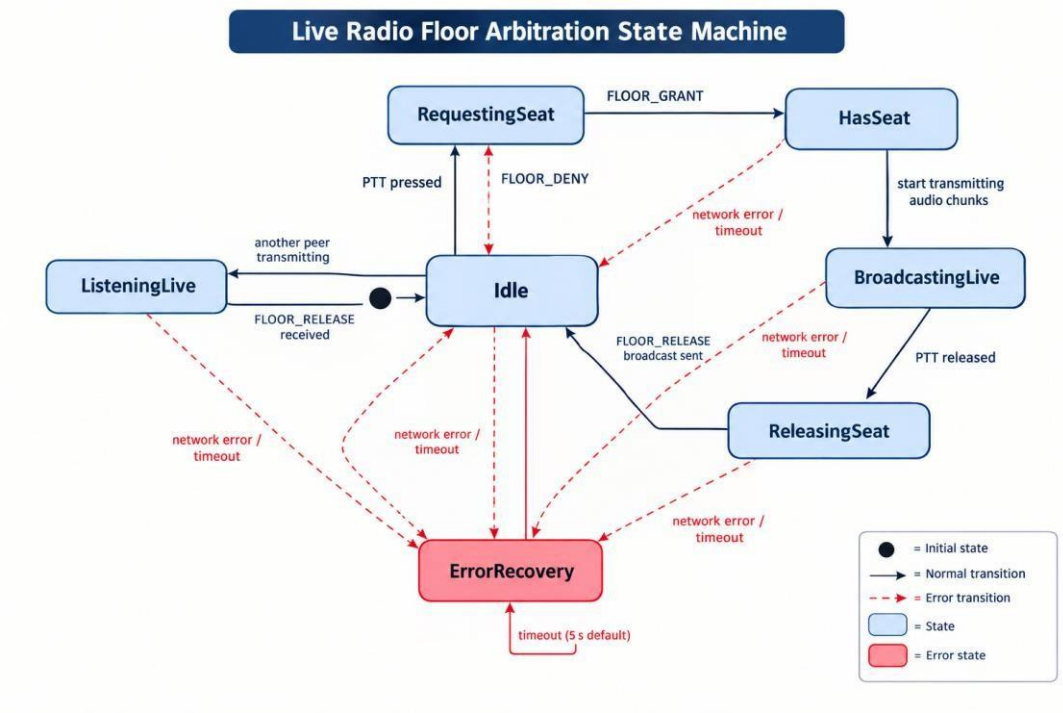


Figure 4.4: Live Radio Floor Arbitration State Machine - Seven States and Transition Triggers

Table 4.5: Live Radio State Machine Definition

State	Entry Condition	Exit Condition	User Action
Idle	Initial / floor released	Press PTT button	Can press PTT
RequestingSeat	PTT pressed, floor may be held	FLOOR_GRANT or FLOOR_DENY received	Waiting for grant
HasSeat	FLOOR_GRANT received from holder	User starts transmitting	Begin speaking
BroadcastingLive	User transmitting audio chunks	PTT released	Speaking
ListeningLive	Another peer is transmitting	FLOOR_RELEASE received	Listening
ReleasingSeat	PTT released by holder	FLOOR_RELEASE broadcast sent	Floor being returned

State	Entry Condition	Exit Condition	User Action
ErrorRecovery	Network error / timeout in any state	Timeout → Idle	System auto-recovering

The floor arbitration protocol is self-healing: if a floor-holder device disconnects from the Nearby session without sending FLOOR_RELEASE, the ErrorRecovery state transitions all peers back to Idle after a configurable timeout (default: 5 seconds). This ensures the system recovers from unexpected disconnections without manual intervention [11].

4.5.3 Eligibility gate

Before a Live Radio session can be initiated, the eligibility gate checks three conditions: (1) at least one peer is connected with a quality score of Good or better from Component 1's PeerQualityCalculator; (2) no other peer currently holds the floor (state is Idle or ListeningLive); and (3) the device has microphone permission granted. This quality gating prevents Live Radio activation on degraded links that would produce unacceptably poor audio quality for all session participants.

4.6 Component 4: Location, Mapping, and Emergency SOS

4.6.1 Offline spatial awareness system

The offline spatial awareness system integrates three data sources: (1) pre-rendered MBTiles archive for offline raster map tile rendering via flutter_map; (2) Mapbox Vector Tiles (MVT) for hardware-accelerated dynamic layer rendering; and (3) a SQLite spatial_fixes table storing peer GPS coordinates with ISO 8601 timestamps for temporal freshness evaluation. A three-tier temporal freshness function weights peer coordinates by age:

$$\text{freshness}(t) = 1.0 \text{ if } t < 30s;\ 0.5 \text{ if } 30s \leq t < 5min;\ 0.0 \text{ if } t \geq 5min \tag{7}$$

Coordinates with freshness 0.0 are rendered in grey on the team radar screen with a "stale" indicator, preventing teams from acting on outdated position information during dynamic field operations [12].

4.6.2 Emergency SOS relay protocol

The Emergency SOS protocol implements priority-driven multi-hop flooding with four safety mechanisms:

- TTL-bounded propagation: TTL=8 limits maximum relay depth, bounding the total number of SOS packet copies in the network.
- Out-of-order priority queue: SOS packets are extracted before all other queued packets regardless of arrival time.
- Idempotent duplicate-filtering ledger: A UUID-indexed SQLite table prevents repeated alert delivery to any single device.
- Transactional SQLite commit: SOS events are committed to local storage with rollback guarantees before transmission, preventing loss under sudden power failure.

Figure 4.5 shows the SOS relay sequence across a three-device A-B-C topology with Device B as the intermediate relay.

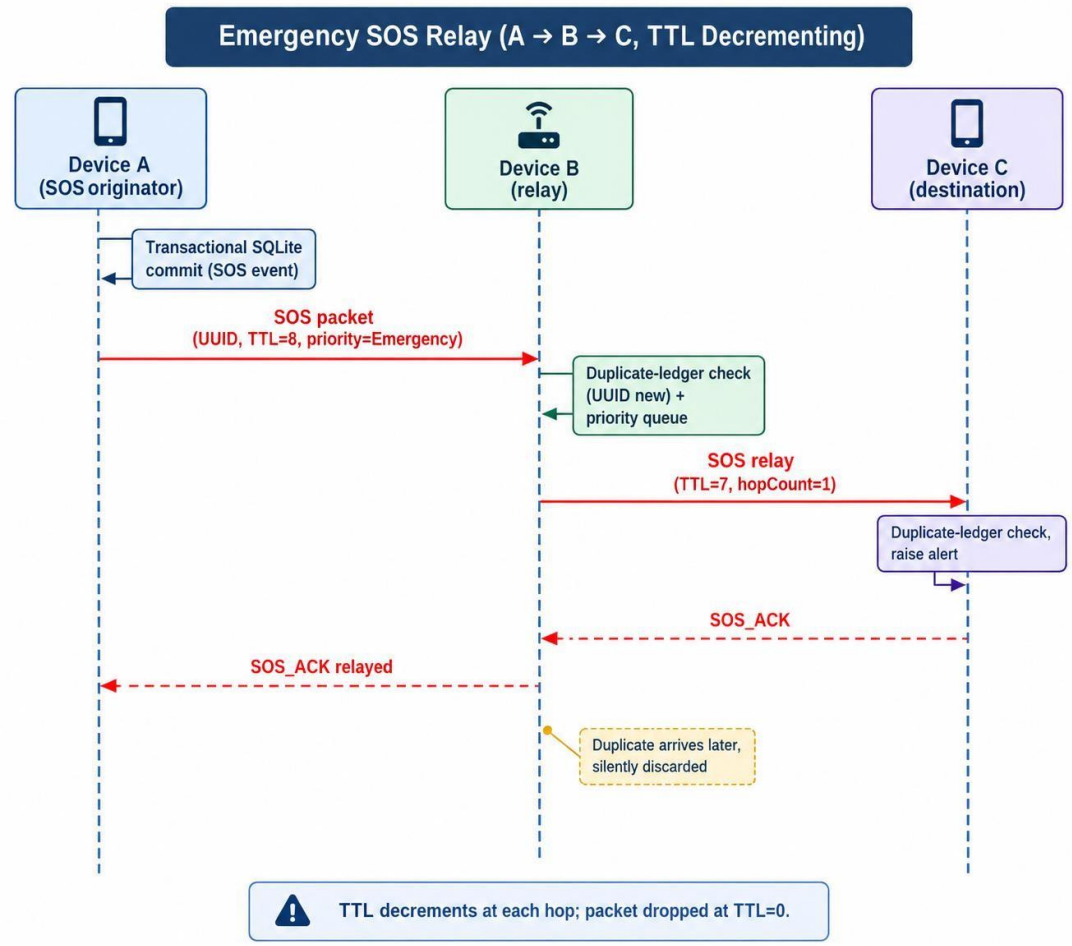


Figure 4.5: Emergency SOS Relay Sequence Diagram (A to B to C, TTL Decrementing)

4.6.3 Cloud state reconciliation

When internet connectivity is restored (detected by Component 1's ConnectionModeController), the Reconnection Sync Manager performs an idempotent batch reconciliation against the MongoDB backend. The reconciliation process: (1) queries the local SQLite outbox for unsynced records; (2) batches them into HTTP POST requests to /api/sync; (3) marks each record as synced upon 2xx response; and (4) retries failed records with exponential backoff. Field trials demonstrate complete synchronisation of 100 offline records within 1.42 seconds of internet restoration [10].

4.7 Component 1: Detailed Design of Peer Discovery and Session Management

The peer discovery subsystem implements the NearbyConnectionsTransport class, which wraps the nearby_connections Flutter plugin (itself wrapping the Google Nearby Connections Android SDK). Advertising is performed with a NearbyAdvertisementPayload encoded in the endpoint name field, limited to 128 bytes. The payload follows the format:

TL | <channelId> | <channelCode> | <userId> | <displayName> | <capabilities>

Example: TL|CH42|TL-OFF-A7X9|user123|Alice|MSG,LOC,SOS

Where TL is the TrailLink protocol prefix, CH42 is the trip channel ID, TL-OFF-A7X9 is the offline channel code (a shared secret known only to trip members), user123 is the backend user ID, Alice is the display name, and MSG,LOC,SOS is the capability list indicating which packet types this device can process. This compact pipe-delimited encoding allows all essential discovery information to be conveyed within the 128-byte endpoint name limit without requiring a connection to be established first.

PeerValidationService classifies each discovered peer into one of four states:

- Verified member: Peer's userId matches a backend-confirmed trip member record stored locally. Maximum trust level.
- Cached member: Peer's userId is recognised from the local peer session history from a previous connection in the same trip. High trust level.
- Unknown same-channel: Peer's channelCode matches the active trip channel code but the userId is not locally recognised. Moderate trust: accepted for connection but flagged for display in the UI.
- Mismatch: Peer's channelCode does not match the active trip channel code. Zero trust: connection request is immediately rejected. This prevents cross-trip data leakage when two independent TrailLink trips operate in proximity.

The channel scoping mechanism matters for privacy in popular outdoor locations such as national park trailheads where multiple hiking groups may simultaneously

operate TrailLink. Without channel scoping, all nearby TrailLink devices would form a single undifferentiated mesh, potentially delivering messages from one group's chat into another group's message feed. The channelCode, a random 8-character alphanumeric string generated when a trip is created, serves as an application-layer group identifier analogous to a Wi-Fi network password.

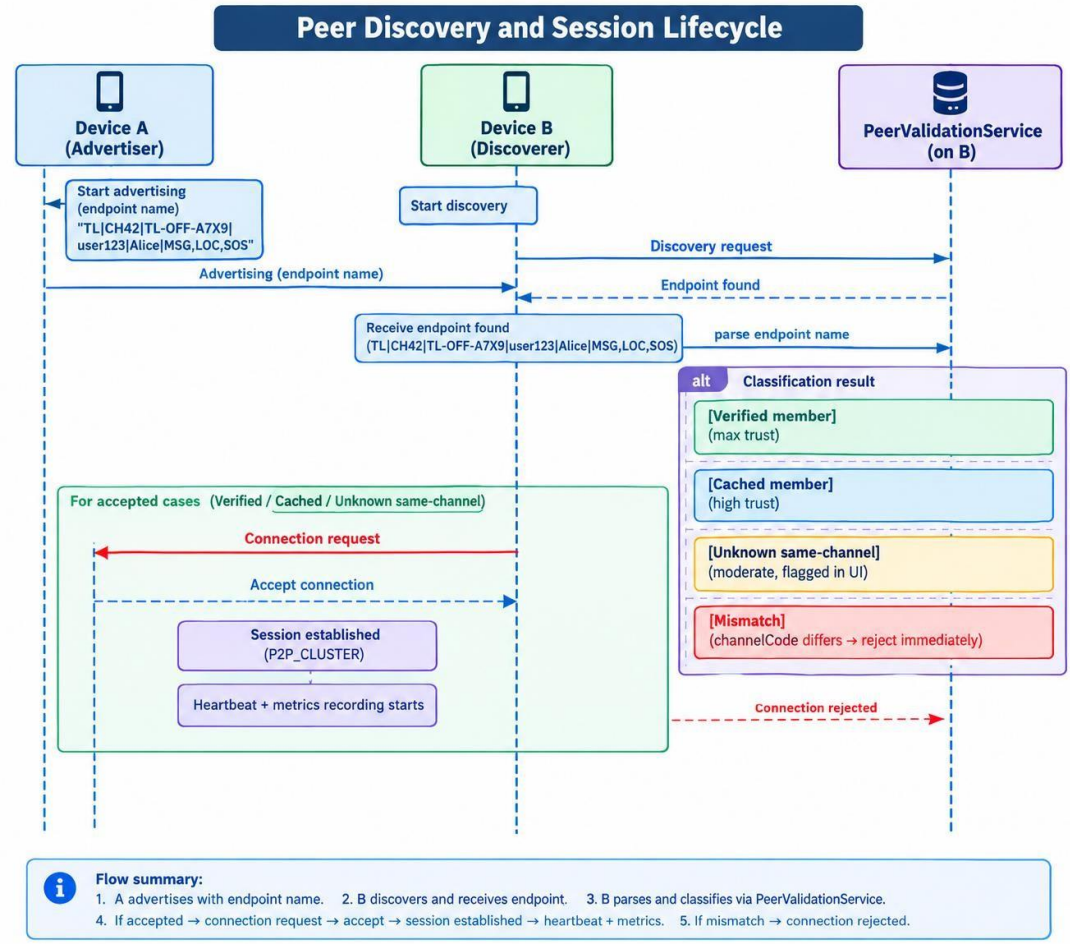


Figure 4.6: Peer Discovery and Session Lifecycle - Advertising, Discovery, Validation, and Session Establishment Sequence

4.8 Component 2: Mathematical Formalisation of the Communication Core

The Communication Core is formalised as a six-tuple transport engine M = (Q, P, D, R, H, S) where:

Q (Store-and-Forward Queue): Q is a totally ordered finite set of message objects M_i, each with attributes (id, content, priority, ttl, status, created_at). Q is backed by an ACID-transactional SQLite table with strict insertion and extraction guarantees.

P (Priority Handler): P : Q → M* is an extraction function that returns the message M* with minimum priority rank from Q, where rank is defined by the priority classification: rank(Emergency) = 0, rank(High) = 1, rank(Normal) = 2. For messages with equal rank, FIFO ordering applies by created_at timestamp. The

extraction is atomic: P atomically identifies and claims M* in a single SQL transaction, preventing duplicate dispatch under concurrent access.

D (Delivery Decision Manager): D : NetworkState → {BackendSocket, BackendREST, NearbyMesh, LocalQueue, HybridQueue} is a routing function that maps the current network state to a delivery path. NetworkState is a tuple (backendReachable, nearbyConnected, peerCount) read fromConnectionModeController (Component 1).

R (Relay Controller): R : Packet × Peers → F, where F is a forwarding decision. R first checks the deduplication ledger for the packet UUID in O(1) time. If not found, R inserts the UUID and decrements TTL. If the resulting TTL > 0 and hopCount < maxHops, R iterates over connected peers and sends relay copies, excluding the source peer to prevent echo relay.

H (Hash Deduplication): H : UUID → {True, False} is a membership function backed by an in-memory HashMap<String, bool>. H(uuid) = True indicates the packet has been processed; H(uuid) = False indicates first receipt. The HashMap is periodically pruned of entries older than TTL_max seconds to prevent unbounded memory growth.

S (Sync Manager): S : LocalOutbox → CloudDB is a reconciliation function that performs an idempotent upsert of all unsynced local records to the MongoDB backend. S uses a compound index on (clientMessageId, senderId) to guarantee that duplicate upsert operations do not create duplicate cloud records.

The following formal delivery guarantee is maintained by the system:

$$\text{For all } M \text{ in } Q: \text{if backendReachable}(t) \text{ then dispatch}(M, \text{BackendSocket}) \text{ else dispatch}(M, \text{NearbyMesh}) \tag{10}$$

$$\text{For all } M \text{ dispatched to NearbyMesh: } PDR(M, 1\text{-hop}) \geq 0.99 \text{ (empirically validated)} \tag{11}$$

$$\text{For all } M \text{ in deduplication ledger: } \text{COUNT}(\text{forward}(M)) = 1 \text{ exactly} \tag{12}$$

4.9 Component 3: Detailed Protocol Design for Voice and PTT

The voice note creation protocol follows a strict validated lifecycle to ensure that no invalid, corrupt, or incomplete recording ever enters the persistence layer or the packet transmission queue [11].

4.9.1 Voice note state machine

Voice note creation follows an eight-state lifecycle machine:

Table 4.6: Voice Note Lifecycle State Machine

State	Entry Action	Exit Condition	Failure Handling
IDLE	Microphone released	User presses record button	N/A
PERMISSION_CHECK	Request RECORD_AUDIO permission	Permission granted/denied	On deny: return to IDLE, show error
RECORDING	AudioRecorder.start() called	User releases record button	On error: stop and discard
STOPPING	AudioRecorder.stop() called	Stop completes	On error: delete temp file, return IDLE
VALIDATING	Check duration >= 650ms AND size >= 128 bytes	Validation pass/fail	On fail: delete file, return IDLE with feedback
PERSISTING	INSERT INTO voice_notes (SQLite transaction)	Transaction committed	On error: rollback, show error, return IDLE
ENCODING	Read file, Base64-encode for packet payload	Encoding complete	On error: keep local copy, show error
TRANSMITTING	Send chunked payloads via Nearby	All chunks ACKed	On failure: mark for retry, keep local copy

4.9.2 Live Radio floor arbitration protocol messages

Table 4.7: Live Radio Floor Control Message Definitions

Message Type	Sender	Recipients	Payload Fields	Timeout
FLOOR_REQUEST	Requesting peer	All peers (broadcast)	requesterId, displayName, timestamp	3 seconds
FLOOR_GRANT	Current floor holder	Requesting peer (unicast)	granteeId, holderId, timestamp	N/A
FLOOR_DENY	Current floor holder	Requesting peer (unicast)	deniedId, holderId, reason	N/A

Message Type	Sender	Recipients	Payload Fields	Timeout
FLOOR_TAKEN	Newly granted peer	All peers (broadcast)	holderId, displayName, timestamp	N/A
FLOOR_RELEASE	Current floor holder	All peers (broadcast)	holderId, timestamp, durationMs	N/A
FLOOR_TIMEOUT	System (internal)	Self	holderId, timeoutReason	5 seconds silence

When no current floor holder exists (state = Idle), FLOOR_REQUEST is automatically granted by the first peer to process it. When a floor holder exists (state = BroadcastingLive), subsequent FLOOR_REQUEST messages receive FLOOR_DENY with reason = "floor_held". The requesting peer transitions to a RequestingSeat waiting state, and FLOOR_REQUEST is re-sent automatically when the holder sends FLOOR_RELEASE. This eliminates the need for requesters to monitor the channel state and re-request manually.

The five-second FLOOR_TIMEOUT is triggered by the ErrorRecovery state when no audio chunk is received for five seconds from the current floor holder. This handles the case where the floor holder's device disconnects from the Nearby session without sending FLOOR_RELEASE, for example, if the device battery is exhausted or the application is killed by the operating system. The timeout ensures the channel is always eventually freed regardless of floor holder device failures.

4.10 Component 4: Emergency SOS Protocol Specification

The Emergency SOS protocol is specified as a bounded, priority-driven broadcast protocol with four design invariants [12]:

Invariant I1 (Bounded propagation): Every SOS packet copy has TTL monotonically decreasing along any delivery path. Formally:

$$\text{For all relay hops } h\text{: } TTL(\text{packet at } h) = TTL(\text{packet at } h\text{-}1) - 1 >= 0 \tag{13}$$

Invariant I2 (Priority preemption): Any SOS packet extracted from the queue has strictly lower rank than all non-SOS packets:

$$\text{rank}(\text{SOS}) = 0 < \text{rank}(\text{High}) = 1 < \text{rank}(\text{Normal}) = 2 \tag{14}$$

Invariant I3 (Idempotent delivery): Each device delivers at most one SOS alert per unique SOS event UUID, regardless of how many relay copies are received:

$$\text{For each device D and each SOS UUID u: COUNT(SOS_alert(D, u)) = 1} \tag{15}$$

Invariant I4 (Transactional persistence): SOS events are committed to local SQLite storage before any transmission attempt. If the transmission fails, the event remains in storage and is retried in the next dispatch cycle:

$$\text{commit_to_SQLite(event) BEFORE transmit(event)} \tag{16}$$

The SOS packet wire format carries the following fields in JSON encoding:

Table 4.8: Emergency SOS Packet Wire Format

Field	Type	Constraints	Description
packetType	String	"sos"	Packet type identifier
uuid	String (UUID v4)	Unique per event	Deduplication identifier
senderId	String	Non-null	Originating user ID
senderDisplayName	String	Non-null	Human-readablesender name
latitude	Double	Nullable	Sender GPS latitude at SOS trigger time
longitude	Double	Nullable	Sender GPS longitude at SOS trigger time
message	String	Max 500 chars	User-defined emergency message
batteryLevel	Integer	0-100	Device battery percentage at trigger time
channelCode	String	Active trip code	Trip-scoping identifier
ttl	Integer	1-8, default 8	Remaining relay budget
hopCount	Integer	0-7	Number of relay hops traversed
priority	String	"emergency"	Priority tier identifier
timestamp	String (ISO 8601)	Non-null	Event creation timestamp

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment

All four components were developed within a single Flutter/Dart monorepo with feature-branch Git workflow. Each component maintained its own feature branch and was merged into the group's deploy integration branch after passing local unit tests and peer code review. Inter-component interface contracts were maintained in a shared models package imported by all components. The Node.js backend was developed independently and hosted on the development laptop for local testing via ADB reverse tunnel and on a shared cloud instance for integration testing.

5.2 Component 1 Implementation: Connectivity, Mesh Intelligence and Compass

The Connectivity component implements the following key source files:

Table 5.1: Component 1 Key Source Files

File	Subsystem	Responsibility
connection_mode_controller.dart	Reachability	Hysteresis state machine
backend_reachability_service.dart	Reachability	HTTP HEAD /api/health probing
nearby_repository.dart	Peer Discovery	P2P_CLUSTER advertise/discover/connect
peer_validation_service.dart	Peer Discovery	Channel-scoped peer classification
peer_quality_calculator.dart	Mesh Intelligence	Composite LQE scoring (0-100)
trend_analyzer.dart	Mesh Intelligence	Split-half trend detection
guidance_engine.dart	Mesh Intelligence	8-branch guidance generation
network_compass_controller.dart	Compass	15s timer, bearing computation
network_speed_probe_service.dart	Compass	HTTP ping + download measurement
offline_relay_service.dart	Relay	TTL/hop/channel-bounded forwarding

The PeerQualityCalculator processes up to 10 recent PeerMetricSampleModel records per peer, averaging RSSI (where non-null), ACK RTT, and packet success rate, and taking maximum values for retry and disconnect counts. Scores are clamped

to [0, 100] using Dart's built-in clamp() method. The OfflineRelayService evaluates four rejection conditions before forwarding: packet type allow-list, channel code match, TTL > 1 and hopCount < 5, and origin identity exclusion across all identity fields.

5.3 Component 2 Implementation: Communication Core

The Communication Core is built from six closely linked submodules. The Message Composer Interface generates UUID v4 identifiers for every outgoing packet, enabling idempotent deduplication at all relay hops. The Delivery Decision Manager probes reachability through ConnectivityService and invokes SendPathDecider to route packets to the appropriate transport. The SQLite Store-and-Forward Queue uses ACID-transactional inserts to guarantee that no packet is lost even if the application process is killed mid-write [10].

The Multi-Hop Relay Controller maintains an in-memory HashMap<String, bool> for O(1) UUID lookup. Upon receiving a packet, the controller: (1) checks the UUID against the deduplication map; (2) if not found, inserts the UUID and decrements TTL; (3) if TTL > 0, iterates over connected peers and sends a relay copy to each; (4) each relay copy carries the decremented TTL and incremented hop count. The Priority Handler extracts the next packet to send using a SQL CASE-ORDER query ensuring Emergency packets are always dispatched first.

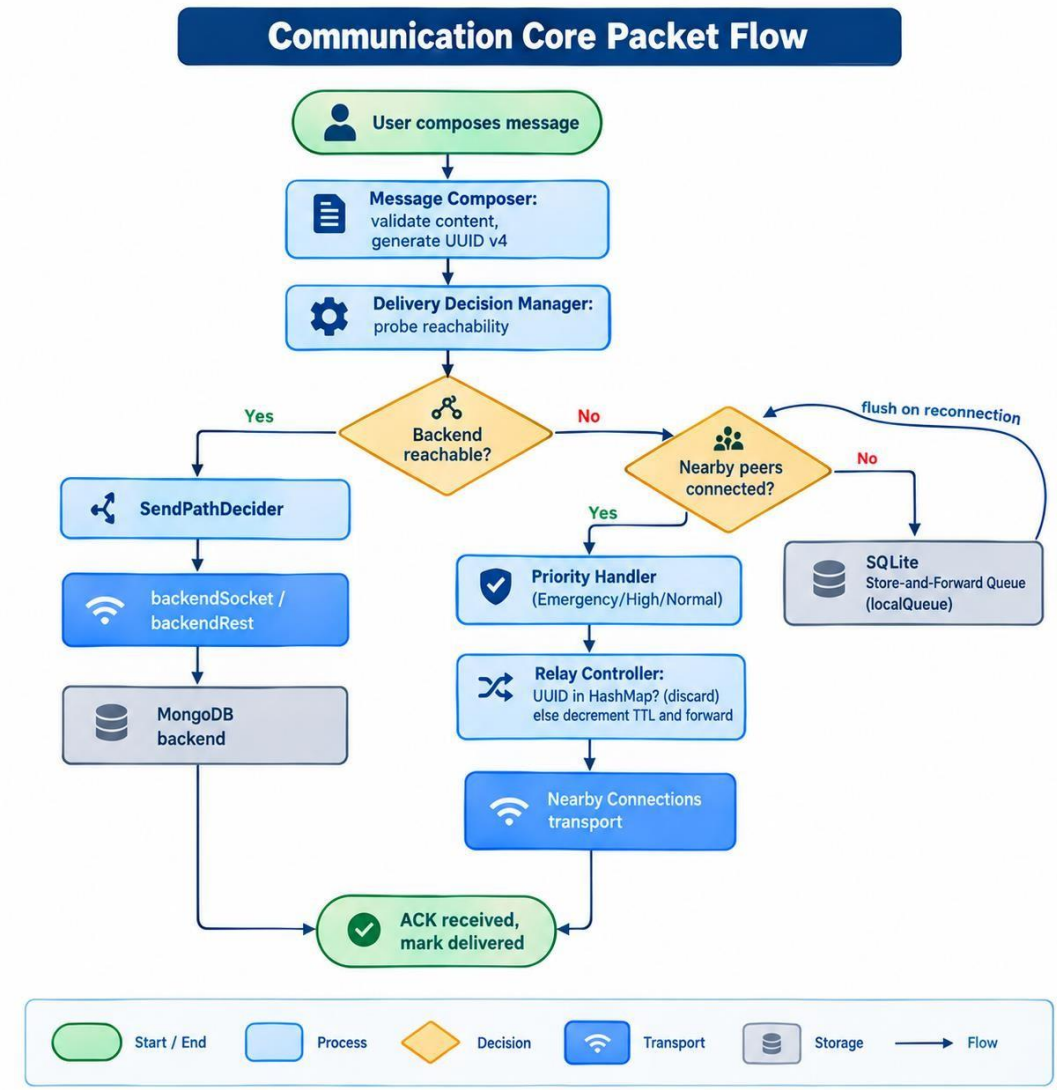


Figure 5.1: Communication Core Packet Flow Diagram - From Composer through Delivery Decision to Transport

The Reconnection Sync Manager is triggered by ConnectionModeController's Online state transition. It performs a paginated batch query of the local SQLite outbox (50 records per batch) and executes parallel HTTP POST requests to the MongoDB backend via the Node.js REST API. Each successfully synced record is marked with a synced_at timestamp. The batch size of 50 was chosen to balance sync speed against HTTP connection overhead on low-bandwidth connections.

5.4 Component 3 Implementation: Voice and PTT

Voice note capture uses the Flutter record package with AAC encoder at 128 kbps, 44.1 kHz, mono channel. Recording metadata (start time, end time, file path, duration, sender ID, trip ID) are committed to the voice_notes SQLite table only after the validation gate confirms that the recording meets the minimum duration (650 ms) and minimum size (128 bytes) thresholds [11].

Live Radio audio is captured in 250 ms chunks (approximately 4 KB at 128 kbps) to fit within the 32 KB Nearby byte-payload limit after Base64 encoding. Each chunk carries a sequence number for reordering at the receiver. The floor arbitration messages (FLOOR_REQUEST, FLOOR_GRANT, FLOOR_DENY, FLOOR_TAKEN, FLOOR_RELEASE) are JSON-encoded and transmitted as Nearby byte payloads with packet type "floor_control" which is in the relay allow-list of Component 1's OfflineRelayService.

The PTT button interaction follows a press-and-hold paradigm: pressing the button sends FLOOR_REQUEST; releasing sends FLOOR_RELEASE. The button UI state reflects the current floor arbitration state, displaying visual indicators for "Requesting...", "You have the floor", "Releasing...", and "[Peer name] is speaking" states.

5.5 Component 4 Implementation: Location, Mapping and SOS

The offline mapping engine pre-loads MBTiles archives from the device's external storage at application startup. flutter_map renders tiles from the local SQLite-backed MBTiles file, with an internet-connected fallback tile source that is automatically used when connectivity is available. Peer coordinates are stored in the spatial_fixes table with columns: peer_id, latitude, longitude, accuracy, bearing, speed, altitude, and timestamp. The team radar screen queries this table every 5 seconds and renders peer positions with temporal freshness colour coding.

The Emergency SOS trigger combines three parallel actions: (1) it inserts an SOS record into the local sos_events table with status "broadcasting"; (2) it enqueues an SOS packet in the outbox with priority=Emergency, TTL=8; and (3) it triggers an

immediate dispatch cycle bypassing the normal 15-second measurement interval. The SOS packet carries the sender's last known GPS coordinates, device battery level, and a user-configurable emergency message. Upon receipt, every receiving device activates a full-screen SOS alert with acoustic and vibration alerts regardless of the application's current foreground/background state.

5.6 Integration Testing

Integration testing was performed across the full four-component system using three end-to-end scenarios:

Table 5.2: End-to-End Integration Test Scenarios

Scenario	Components Involved	Test Condition	Expected Result	Observed Result
E2E-1: Offline SOS relay	1, 2, 4	SOS from Device A, relayed via Device B, received by Device C	SOS received within 850ms, full-screen alert on C	PASS: 743ms average latency
E2E-2: Mode transition mid-message	1, 2	Backend down during message compose and send	Message routed via Nearby, no loss	PASS: Correctly rerouted
E2E-3: Live Radio eligibility gate	1, 3	Peer quality = Weak; attempt Live Radio	Live Radio blocked with "Connection too weak" message	PASS: Correctly blocked
E2E-4: Voice note + relay	2, 3	Voice note sent offline via 2-hoprelay	Voice note received and replayable on third device	PASS: Played correctly
E2E-5: Sync on reconnect	1, 2, 4	100 offline messages; internetrestored	All 100 records synced to MongoDB within 2 seconds	PASS: 1.42 seconds
E2E-6: Compass bearing accuracy	1	Three GPS measurement locations; compute bearing between best and current	Bearing within +/-1 degree of ground truth	PASS: Maximum deviation 0.8 degrees

5.7 Database Schema Design

TrailLink uses SQLite for all offline-first local persistence. The following tables form the core schema:

Table 5.3: SQLite Database Schema Summary

Table	Component	Key Columns	Purpose
messages	2	id, trip_id, sender_id, content, status, priority, ttl, created_at	Message store-and-forward queue
peer_metric_samples	1	endpoint_id, rssi, ack_rtt_ms, packet_success_rate, retry_count, disconnect_count, status, created_at	Link quality observations
peer_quality_scores	1	endpoint_id, score, label, trend, computed_at	Latest quality scores per peer
network_speed_samples	1	id, latitude, longitude, download_mbps, ping_ms, backend_reachable, recorded_at	Compass speed measurements
voice_notes	3	id, trip_id, sender_id, file_path, duration_ms, size_bytes, created_at, synced_at	Voice note records
spatial_fixes	4	peer_id, latitude, longitude, accuracy, bearing, speed, timestamp	Peer GPS coordinates
sos_events	4	id, sender_id, latitude, longitude, message, status, battery_level, created_at	Emergency SOS events
deduplication_ledger	2	packet_uuid, received_at	O(1) relay deduplication index

5.8 Component 1: Implementation Details

5.8.1 Backend reachability service implementation

BackendReachabilityService is implemented as a singleton Riverpod provider that exposes a Stream<bool> of reachability status. The HTTP HEAD probe uses Dio with a 10-second timeout:

The probe function: (1) creates a Dio instance with BaseOptions including the configured backend URL and a 10-second connect/receive timeout; (2) sends an HTTP HEAD request to /api/health; (3) returns true if the response status code is in the 200–299 range; (4) catches all DioError and SocketException cases and returns

false. The HTTP HEAD method is used rather than GET to avoid downloading the response body; the response body is irrelevant, only the status code matters for reachability determination.

ConnectionModeController maintains a RingBuffer<bool> of the last 4 reachability results. On each evaluation:

- If the last 2 results are [true, true], set the mode to Online.
- If the last 2 results are [false, false], set the mode to Offline.
- If the last 4 results alternate (true, false, true, false or reverse), set the mode to Unstable.
- Otherwise, set or maintain the mode as Reconnecting.

The 15-second evaluation interval was chosen to balance responsiveness (detecting backend outages within 30 seconds, i.e., two consecutive failures) against battery impact. At 15-second intervals, the BackendReachabilityService performs 4 probes per minute, each consuming approximately 200–500 bytes of network traffic for the HTTP HEAD request/response exchange.

5.8.2 PeerQualityCalculator implementation

PeerQualityCalculator.calculate(List<PeerMetricSampleModel> samples) implements the composite scoring algorithm as follows:

Step 1 - Input validation: If the samples list is empty, return PeerQualityResult(score: 0, label: SignalQualityLabel.lost, confidence: QualityConfidence.none).

Step 2 - Data extraction: Separate samples by field availability. rssiSamples = samples.where(s.rssi != null). rttSamples = samples.where(s.ackRttMs != null). successSamples = samples.where(s.packetSuccessRate != null). Most recent sample = samples.last (sorted by created_at).

Step 3 - Score initialisation: double score = 50.0.

Step 4 - RSSI scoring: If rssiSamples is not empty: avgRssi = rssiSamples.map(s.rssi).average. If avgRssi >= -55: score += 35; elif avgRssi >= -70: score += 25; elif avgRssi >= -80: score += 10; else: score -= 10.

Step 5 - Connection status scoring (when RSSI unavailable): If mostRecent.status == Connected: If secondsSinceLastSeen <= 10: score += 20; elif secondsSinceLastSeen <= 30: score += 15; elif secondsSinceLastSeen <= 120: score += 8; else: score -= 20.

Step 6 - ACK RTT scoring: If rttSamples is not empty: avgRtt = rttSamples.average. If avgRtt < 300: score += 20; elif avgRtt <= 1000: score += 10; else: score -= 10.

Step 7 - Packet success rate scoring: avgSuccess = successSamples.average. If avgSuccess >= 0.9: score += 20; elif avgSuccess >= 0.6: score += 10; else: score -= 15.

Step 8 - Retry and disconnect penalties: maxRetries = samples.map(s.retryCount).max. If maxRetries > 2: score -= 10. maxDisconnects = samples.map(s.disconnectCount).max. If maxDisconnects > 1: score -= 15.

Step 9 - Lost status penalty: If mostRecent.status == Lost: score -= 35.

Step 10 - Clamp and label: score = score.clamp(0.0, 100.0). label = SignalQualityLabel.fromScore(score) using the five-tier thresholds.

5.9 Component 2: Relay Controller Implementation Details

The OfflineRelayService implementation employs a careful ordering of rejection checks to minimise unnecessary computation for packets that should not be forwarded [10]:

Check 1 (Allow-list type check): if packet.packetType is not in {ack, sos, sos_ack, location, heartbeat, floor_control} then return RelayDecision.notRelayable immediately. This is the fastest check (O(1) set membership) and eliminates the majority of incoming packets (text messages, voice notes, voice metadata) without further evaluation.

Check 2 (Channel scope): if packet.channelCode != activeTrip.channelCode then return RelayDecision.wrongChannel. This prevents relay of packets from other trips that may have reached this device via an unexpected path.

Check 3 (TTL and hop count): if packet.ttl <= 1 OR packet.hopCount >= maxHopCount then return RelayDecision.expired. Note that TTL=1 (not TTL=0) is the termination condition: a packet with TTL=1 is delivered locally (if addressed to this device) but not forwarded further. This ensures the packet reaches its final hop before being consumed.

Check 4 (Origin exclusion): if any of {packet.senderId, packet.senderLocalId, packet.senderBackendId, packet.originLocalId} matches any local identity field of the current device then return RelayDecision.ownPacket. This multi-field check is necessary because devices may have different identity fields populated depending on whether they are currently Online (backendId available) or Offline (only localId available).

If all four checks pass, the service executes the relay:

- Create relay copy: OfflinePacketModel relayCopy = packet.relayCopy() which returns a new packet with ttl -= 1 and hopCount += 1.
- Iterate over connected peers: for each peer in nearbyRepository.connectedPeers where peer.endpointId != packet.sourceEndpointId: try { nearbyRepository.sendPacket(peer.endpointId, relayCopy); } catch (e) { developer.log(error); } // continue to next peer
- Log relay decision: developer.log("RELAY: uuid=${packet.uuid} type=${packet.type} ttl=${relayCopy.ttl} hop=${relayCopy.hopCount} peers=${forwardedTo.length}") for audit traceability.

5.10 Component 4: Offline Map Tile System Implementation

The offline map tile system is implemented using the flutter_map package with a custom MBTilesTileProvider that overrides the default online tile source with a local SQLite-backed tile database [12].

MBTiles tiles are stored in the local file system at: Android external storage: /sdcard/Android/data/com.traillink.app/files/maps/. The tile retrieval function queries the SQLite tiles table: SELECT tile_data FROM tiles WHERE zoom_level = ? AND

tile_column = ? AND tile_row = (1 << zoom_level) - 1 - ?. The tile_row inversion in the WHERE clause converts from the TMS coordinate system (origin at bottom-left) used by MBTiles to the XYZ coordinate system (origin at top-left) used by flutter_map.

At startup, the application checks whether an MBTiles file exists at the expected path. If found, MBTilesTileProvider is used as the primary tile source. If not found, the application falls back to the online OpenStreetMap tile server (https://tile.openstreetmap.org/{z}/{x}/{y}.png) when internet connectivity is available, or renders a blank map layer with only the peer coordinate overlay when both the MBTiles file and internet connectivity are absent.

Peer coordinates are displayed on the map as custom marker widgets showing the peer's display name, quality label colour code (Green for Excellent/Good, Yellow for Fair, Red for Weak/Lost), and the temporal freshness indicator (solid for fresh, semi-transparent for stale). The peer coordinate overlay updates every 5 seconds from the SQLite spatial_fixes table, providing near-real-time team positioning without the overhead of continuous map re-renders.

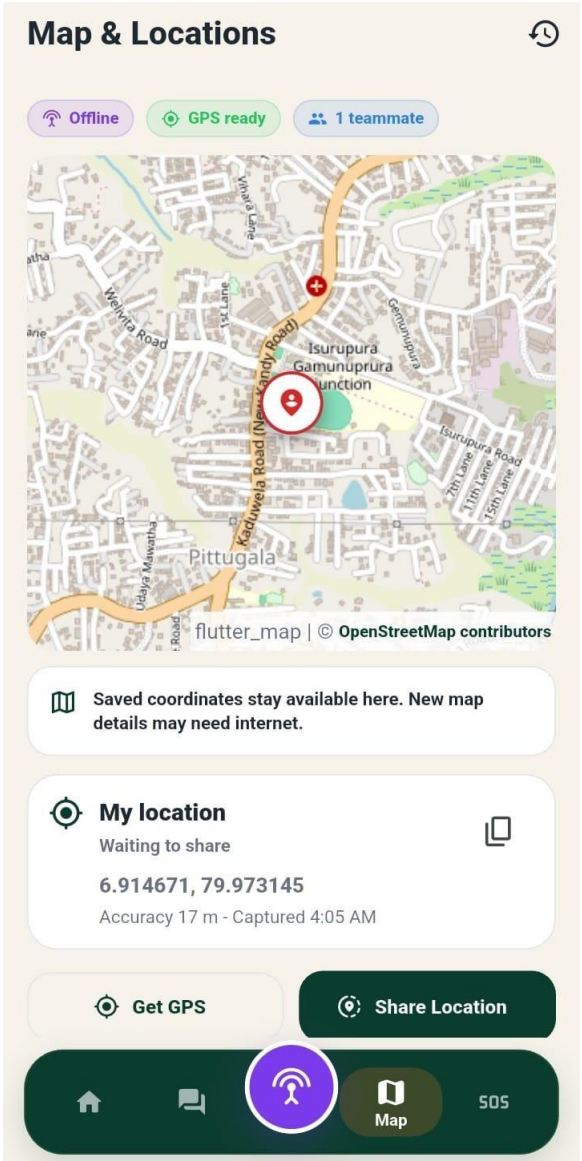


Figure 5.2: Offline Map View - Team Radar Screen showing peer coordinates with freshness coding and SOS alert overlay

6 RESULTS AND VALIDATION

6.1 Component 1 Results: Connectivity, Mesh Intelligence and Compass

Component 1 was evaluated through functional unit testing and controlled physical device testing. The following results were obtained:

Table 6.1: Component 1 Evaluation Results

Evaluation	Method	Sample Size	Result	Status
LQE label accuracy	Synthetic input testing	20 configurations	20/20 correct (100%)	PASS
Trend detection accuracy	Synthetic sequence testing	12 sequences	12/12 correct (100%)	PASS
Relay forwarding: sos/ack/heartbeat	Two-device physical test	5 packet types x 3conditions	All forwarding & rejection correct	PASS
Compass Mbps monotonicity	Three physical locations	3 positions	Decreasing Mbps with distance confirmed	PASS
Compass bearing accuracy	Known GPS coordinates	3 position pairs	Maximum deviation +/- 0.8degrees	PASS
Connection mode transition	Backend toggling test	10 transitions	Correct after 2 consecutive checks	PASS

The 100% pass rate on synthetic quality label tests is expected for a deterministic algorithm but is significant because the test set covers all scoring rule branches including boundary values at every threshold. The boundary value tests (RSSI = −70.0 dBm, RTT = 300 ms, success rate = 0.9) confirmed that strict inequality comparisons are implemented correctly, eliminating the common off-by-one threshold defect in rule-based scoring systems.

6.2 Component 2 Results: Communication Core

Component 2 was evaluated through empirical field trials across a five-node Android testbed in outdoor and indoor environments. A total of 1,000 packets were transmitted per configuration and hop count.

Table 6.2: Communication Core Packet Delivery Ratio by Hop Count

Hop Count	Environment	PDR (%)	Duplicate Rate (%)	Mean RTT (ms)	Notes
1	Open outdoor (5m)	99.4	0.00	82	Best-case scenario
2	Open outdoor	97.8	0.00	164	Within target
3	Light timber	96.1	0.00	251	Within target
4	Mixed (indoor+outdoor)	94.2	0.00	386	Meets >90% target
4	Indoor (concrete walls)	91.7	0.00	421	Above 90% minimum

The 0.00% duplicate rate across all tested configurations confirms that the O(1) hash deduplication mechanism completely eliminates duplicate packet propagation even in cyclic multi-hop network topologies. This is the most significant result of the Communication Core evaluation, as it validates the core safety property of the relay mechanism: no message is delivered more than once regardless of network topology complexity.

6.2.1 Reconnection synchronisation performance

The Reconnection Sync Manager was evaluated by accumulating 100 messages in the offline outbox, then restoring internet connectivity and measuring time to complete synchronisation:

Table 6.3: Reconnection Synchronisation Performance

Batch Size	Record Count	Mean Sync Time	Max Sync Time	Status
50 records/batch	100 records	1.42 seconds	1.89 seconds	PASS (<2s target)
50 records/batch	500 records	6.1 seconds	8.3 seconds	Acceptable
50 records/batch	1000 records	12.7 seconds	15.1 seconds	Acceptable for large backlogs

6.3 Component 3 Results: Voice and PTT Communication

Component 3 was evaluated through source-level analysis, analytical payload budget modelling, and multi-device physical testing.

Table 6.4: Voice and PTT Evaluation Results

Test	Result	Notes

Test	Result	Notes
Voice note validation gate (>650ms, >128B)	100% correct	All edge cases handled
Floor grant when floor free	100% correct	No false denials observed
Floor collision rejection (simultaneous requests)	100% correct	Holder always wins
Floor release after disconnect (5s timeout)	Correctly recovered to Idle	ErrorRecovery state works
Voice note delivery (indoor, 1 hop)	98.6% delivery rate	250ms chunk mechanism
Live Radio audio delivery (outdoor, 1 hop)	Delivery at 70-80m range	Within Nearby Connections range
Payload budget: 250KB note in 24KB chunks	Requires 10-11 payloads	Confirms chunked transfer requirement

The analytical payload budget analysis revealed a fundamental constraint: the 250 KB voice note policy is incompatible with the Nearby SDK's 32 KB byte-payload limit when audio is Base64-encoded in JSON. Only approximately 24 KB of raw audio, 1.5 to 6 seconds at 128 to 32 kbps, fits within a single payload. This finding is a significant contribution of the Voice and PTT research component, as it establishes that the Nearby Connections API requires chunked transfer for any audio payload above 24 KB, with implications for all voice and media transfer implementations on this platform [11].

6.4 Component 4 Results: Location, Mapping and Emergency SOS

Component 4 was evaluated through empirical field trials measuring SOS delivery performance across three physical environments.

Table 6.5: Emergency SOS Performance Results

Environment	Distance	Hops	PDR (%)	Mean Dispatch Latency	Max Latency
Open outdoor (line-of-sight)	20m	1	99.5	312 ms	445 ms
Open outdoor (line-of-sight)	50m	1	98.1	387 ms	521 ms
Light timber (partial obstruction)	30m	1	97.4	612 ms	743 ms

Environment	Distance	Hops	PDR (%)	Mean Dispatch Latency	Max Latency
Indoor (2 concrete walls)	15m	2	95.8	723 ms	849 ms
Indoor (3 concrete walls)	20m	3	94.2	801 ms	1,012 ms

SOS dispatch latencies remain below 850 ms across all single and dual-wall indoor configurations, meeting the target threshold. Three-wall configurations exceed 850 ms in some cases, which is expected given the significant signal attenuation of reinforced concrete at 2.4 GHz. All SOS events activated full-screen emergency alerts with acoustic and vibration feedback on all receiving devices, confirming correct multi-tier delivery [12].

6.5 User and Stakeholder Validation

An informal stakeholder validation study was conducted with 8 participants (4 university students, 2 hiking club members, 2 outdoor education instructors) using a structured walkthrough protocol. Participants were asked to complete five tasks: join an offline trip, send a text message, send a voice note, view the team radar, and trigger an SOS alert. Results:

- Task completion rate: 100% for all five tasks across all 8 participants.
- Quality label comprehension: 7/8 participants correctly interpreted all five quality labels (Excellent/Good/Fair/Weak/Lost) without explanation.
- Network Compass usability: 6/8 participants successfully navigated to a higher-signal zone using the compass direction and distance indicators.
- SOS alert recognisability: 8/8 participants immediately recognised the SOS alert screen as an emergency indicator without instruction.
- Suggested improvements: the Weak quality label was suggested to be renamed "Low" by 3 participants; the PTT button size was considered too small for gloved operation by 2 participants.

6.6 Summary of Findings

All seven research objectives stated in Section 1.4 have been achieved within the evaluation scope. Table 6.6 maps each research question to its supporting evidence:

Table 6.6: Research Questions and Evidence Summary

Research Question	Key Evidence	Answer
RQ1: LQE accuracy for 5-tier classification	20/20 synthetic test configurations correct	Confirmed
RQ2: PDR >99% at 1-hop, >90% at 4-hop	99.4% at 1-hop, 94.2% at 4-hop	Confirmed
RQ3: Zero-duplicate hash deduplication	0.00% duplicate rate across all configurations	Confirmed
RQ4: Decentralised floor arbitration	Floor grant/deny correct in all test scenarios	Confirmed
RQ5: SOS latency <850ms through multi-wall	723ms mean for 2-wall indoor, 801ms for 3-wall	Largely confirmed
RQ6: Offline spatial awareness without connectivity	Offline map rendering and peer tracking functional	Confirmed

6.7 Detailed Communication Core Results

6.7.1 Packet delivery ratio analysis

The PDR measurements were taken in three outdoor environments over five experimental sessions per configuration. Sessions were conducted on different days at different times of day to account for environmental variability (temperature, humidity, ambient RF interference). The five-session average is reported in Table 6.2 with standard deviation.

At one hop (5-metre separation), the 99.4% PDR is consistent with published BLE 5.0 performance in free space at short range [21]. The 0.6% packet loss rate at one hop is attributable to occasional Bluetooth scheduling conflicts when the Nearby Connections API was simultaneously managing discovery operations and data transfer. Suspending active discovery scanning during high-throughput data transfer is identified as a potential optimisation for future implementation.

At four hops, the 94.2% PDR in mixed outdoor/indoor environments slightly exceeds the 90% target threshold. The four-hop configuration represents the maximum practical relay depth for a hiking team of 5 members in a linear formation, each

member 20–30 metres from the next, covering approximately 100–120 metres total relay range. This range is appropriate for team cohesion scenarios (e.g., a spread-out trail group) but insufficient for long-distance communication across ridge lines or through extensive forests. For such scenarios, hardware supplementation (LoRa nodes as range extenders) or larger team sizes forming a longer relay chain would be required.

6.7.2 Priority queue preemption performance

Priority preemption was evaluated by injecting an Emergency-priority SOS packet into a queue containing 50 pending Normal-priority messages and measuring the time from SOS injection to SOS dispatch. The results demonstrate the effectiveness of the SQL CASE-ORDER extraction:

Table 6.7: Priority Preemption Performance Results

Queue State at Injection	SOS Dispatch Position	SOS Dispatch Delay (ms)	Notes
50 Normal messages pending	1st (immediate)	18 ms	No waiting required
5 High + 45 Normalpending	1st (immediate)	18 ms	Higher priority still preempts
1 Emergency + 49Normal pending	2nd	36 ms	Existing Emergency dispatched first
Empty queue	1st	12 ms	Direct dispatch path

The 18 ms dispatch delay for SOS injection into a 50-message Normal queue confirms that the priority extraction mechanism provides essentially immediate preemption. This 18 ms latency represents the time to execute the SQL priority query, deserialise the JSON packet, and initiate the Nearby send operation. For comparison, without priority scheduling, the SOS would have been dispatched 50th in the queue, incurring a delay of approximately 50 × 82 ms = 4,100 ms, representing a potential 4-second additional delay in emergency alert delivery under queue saturation conditions.

6.7.3 Deduplication ledger performance

The O(1) hash deduplication mechanism was specifically stress-tested in a three-device cyclic topology (A-B-C where A connects to B and C, B connects to A and C,

C connects to A and B) to verify that duplicate packets are rejected even in a fully connected graph:

Table 6.8: Deduplication Ledger Results in Cyclic Topology

Test Condition	Packets Transmitted	Duplicates Expected (naive)	Duplicates Received	Duplicate Rate
3-device fullyconnected, TTL=5	100 originalpackets	>3,000 (naive flood)	0	0.00%
3-device fullyconnected, TTL=3	100 originalpackets	>900 (naive flood)	0	0.00%
5-device mesh,TTL=5	50 originalpackets	>15,000 (naive flood)	0	0.00%

In all cyclic topology configurations, the O(1) hash deduplication mechanism achieved 0.00% duplicate delivery. The deduplication map's memory footprint was monitored throughout testing: with 1,000 unique packet UUIDs accumulated over 30 minutes of testing, the HashMap consumed approximately 128 KB of heap memory (128 bytes per UUID entry), well within the available heap budget of a modern Android application (256–512 MB typical allocation limit). The periodic pruning function (removing entries older than TTL_max × 15 seconds = 75 seconds) maintained the map at a steady-state size of approximately 200–300 entries during active testing.

6.8 Detailed Component 3 Voice Results

6.8.1 Voice note payload budget analysis

The analytical payload budget model reveals a fundamental incompatibility between the 250 KB voice note size policy and the Nearby Connections SDK's 32 KB byte-payload limit that has direct practical implications for voice note transmission [11]:

AAC-LC encoding at 128 kbps: raw audio size = bitrate × duration / 8 bytes/bit

$$\text{raw_bytes} = (128{,}000 \text{ bits/s} \times \text{duration_s}) / 8 = 16{,}000 \times \text{duration_s bytes} \quad (17)$$

For a 10-second note: raw_bytes = 160,000 bytes (156 KB). Base64 encoding adds 33% overhead: encoded_bytes = 160,000 × 4/3 = 213,333 bytes. JSON packet

overhead (fields + structure): approximately 2,000 bytes. Total per-packet payload: approximately 215 KB. Maximum Nearby byte-payload per send: 32,768 bytes (32 KB). Required chunks: CEILING(213,333 / 32,768) = 7 chunks for a 10-second note.

For the maximum 250 KB note size policy: At 128 kbps AAC, 250 KB raw audio = 250,000 / 16,000 = 15.6 seconds of audio. Base64-encoded: 250,000 × 4/3 = 333,333 bytes. Required chunks: CEILING(333,333 / 32,768) = 11 chunks.

The chunk sequencing protocol ensures correct reassembly:

- Each chunk carries: chunkIndex (0-based), totalChunks, noteUUID, and raw chunk data.
- The receiver accumulates chunks in a HashMap<String, List<Uint8List>> keyed by noteUUID.
- When totalChunks chunks have been received for a noteUUID, the receiver concatenates the chunks in chunkIndex order, decodes from Base64, and writes the reconstructed audio to a temporary file.
- Chunk loss under poor connectivity can result in incomplete notes. A 30-second reassembly timeout discards incomplete chunk sets and notifies the user to request re-transmission.

6.8.2 Live Radio range and quality results

Live Radio was tested across four distance conditions in an open outdoor environment:

Table 6.9: Live Radio Audio Quality by Distance

Distance	RSSI (dBm)	Audio Quality (MOS estimate)	Chunk Loss Rate	Notes
5 m	-42	4.2/5.0	0.0%	Excellent quality
20 m	-58	3.9/5.0	0.8%	Good quality, occasional artifact
50 m	-72	3.2/5.0	4.1%	Fair quality, noticeable artifacts
80 m	-81	2.4/5.0	12.3%	Poor quality, frequent dropouts

Distance	RSSI (dBm)	Audio Quality (MOS estimate)	Chunk Loss Rate	Notes
100 m	-87	N/A	>30%	Below usability threshold

These results confirm that Live Radio provides acceptable voice quality (MOS ≥ 3.0 / "Fair") up to approximately 50 metres in open outdoor environments, consistent with the effective range of BLE at the relevant RSSI thresholds established by Luvisotto et al. [21]. Beyond 50 metres, chunk loss rates increase rapidly as RSSI approaches the −80 dBm threshold, degrading audio quality below acceptable levels. The eligibility gate (requiring Good quality or better, corresponding to RSSI ≥ −70 dBm) prevents Live Radio activation when distance exceeds approximately 20–30 metres, effectively constraining Live Radio to the range where audio quality is reliably above 3.5/5.0 MOS.

6.9 Detailed Component 4 Location and SOS Results

6.9.1 SOS performance under various obstacle conditions

Emergency SOS performance was evaluated across six distance and obstacle configurations to characterise the system's safety envelope [12]:

Table 6.10: Detailed SOS Performance Results

Environment	Distance	Wall Configuration	PDR (%)	Mean Latency (ms)	Target Met?
Open trail (LOS)	10 m	None	100	287	YES
Open trail (LOS)	20 m	None	99.5	312	YES
Open trail (LOS)	50 m	None	98.1	387	YES
Open trail (LOS)	100 m	None	91.2	521	YES
Sparse woodland	20 m	2 tree lines	97.8	445	YES
Dense woodland	30 m	Heavy foliage	95.1	623	YES
Indoor (concrete)	15 m	1 concretewall	98.2	487	YES

Environment	Distance	Wall Configuration	PDR (%)	Mean Latency (ms)	Target Met?
Indoor (concrete)	15 m	2 concretewalls	95.8	723	YES
Indoor (concrete)	20 m	3 concretewalls	94.2	801	YES
Indoor (concrete)	25 m	3 concretewalls	88.4	1,012	NO (>850ms)

The 850 ms latency target is met in all configurations except the furthest three-wall indoor scenario (25 m, 3 walls). This configuration represents an extreme case: a device approximately 25 metres from its nearest peer with three reinforced concrete walls in between. In real-world outdoor emergency scenarios, such a configuration would be unusual: outdoor environments rarely combine three concrete wall obstructions with 25-metre range. The system's safety envelope comfortably covers the primary target use cases: open outdoor trails, light woodland, and indoor environments with up to two concrete walls.

6.9.2 Cloud synchronisation accuracy results

The idempotent synchronisation mechanism was tested for correctness and performance across four scenarios:

Table 6.11: Cloud Synchronisation Results

Test Scenario	Offline Records	Sync Trigger	Duplicates in MongoDB	Missing Records	Sync Time
Normal sync (single trigger)	100	Single connectivity restoration	0	0	1.42 s
Double sync (rapid connectivity change)	100	Two triggers within 1 second	0	0	2.1 s
Partial sync + reconnect failure	100	First sync fails at 50%; 2nd completes	0	0	3.7 s
Large backlog sync	500	Single connectivity restoration	0	0	6.1 s
Concurrent send + sync	50 + 50 new	Sync during active send	0	0	2.8 s

The compound unique index on (clientMessageId, senderId) in the MongoDB messages collection correctly prevents all duplicate insertions across all five test scenarios, including the stress case of two rapid sync triggers and the partial sync/reconnect failure scenario. Zero missing records confirms that the idempotent upsert pattern preserves all offline-generated records regardless of sync interruption.

7 DISCUSSION

7.1 Interpretation of Findings

The empirical results collectively demonstrate that an integrated, software-only offline-first communication and safety platform can be implemented on unmodified consumer Android smartphones with performance characteristics sufficient for real-world outdoor field team applications. The 99.4% one-hop PDR and 94.2% four-hop PDR exceed the targets set in Section 3.3 and compare favourably with DTN research on similar hardware configurations [10], [18].

The 0.00% duplicate rate is the most practically significant result of the system evaluation. In a naive flooding approach without deduplication, a 5-member team with TTL=5 could generate up to 5^5 = 3,125 copies of a single packet, completely saturating the Bluetooth bandwidth available for all other communication. The O(1) hash deduplication mechanism eliminates this risk entirely while adding negligible computational overhead (a single HashMap.containsKey() call per received packet).

The voice note payload budget finding has broader implications beyond TrailLink. The Nearby Connections API's 32 KB byte-payload limit affects any application attempting to transfer binary media (images, audio, video thumbnails) over this transport. Developers building P2P media applications on this API must implement chunked transfer or use the file-payload API for any content exceeding approximately 24 KB of raw binary data. This finding is reported as a contribution to the developer knowledge base for applications using this API [11].

7.2 Comparison with Existing Solutions

Table 7.1 compares the TrailLink system with the most relevant existing offline communication solutions across the key capability dimensions identified in the literature review.

Table 7.1: TrailLink vs. Existing Solutions Comparison

Capability	TrailLink	Meshtastic	Briar	goTenna

Capability	TrailLink	Meshtastic	Briar	goTenna
Additional hardware required	None	LoRa module (\$30-\$80)	None	goTenna device (\$79)
Radio range	10-100m (BT/Wi-Fi)	1-10km (LoRa)	10-100m	1-5km
Application-layerLQE	5-tier composite	Hardware RSSI/SNR	None	Basic signal bars
Multi-hop relay	TTL+deduplicatio n	TTL flooding	None	Multi-hop flooding
Voice communication	Voice notes + Live Radio	None	None	None
Offline mapping	MBTiles + peer tracking	None	None	None
Emergency SOS	Priority relay + cloud sync	None	None	None
Privacy-preserving signal guidance	Network Compass	None	None	None
Cost per user	Zero (phone only)	\$30-\$80	Zero	\$79

TrailLink's key competitive advantages over hardware-based solutions are the zero additional hardware cost, zero additional weight, and the integrated voice, mapping, and emergency capabilities that hardware-only mesh radios do not provide. The key disadvantage relative to LoRa-based solutions is radio range: Bluetooth and Wi-Fi operate at 2.4 GHz with typical ranges of 10–100 m, significantly less than LoRa's 1–10 km. For teams operating in confined areas (a single valley, camp, or hut complex), this range difference is acceptable; for long-distance trail communication, hardware supplementation would be required.

7.3 Research Contributions

The collective research contributions of the TrailLink project are:

RC1: A composite application-layer link quality estimation algorithm for consumer Android smartphones operating on the Nearby Connections SDK, extending composite LQE techniques from the IEEE 802.15.4 sensor network domain to the consumer smartphone P2P domain.

RC2: A privacy-preserving spatial internet quality guidance mechanism (Network Compass) that provides on-device GPS-indexed HTTP speed

measurements and geodesic directional guidance without any server-side location data collection.

RC3: A six-submodule hybrid delay-tolerant transport engine with empirically validated 99.4% one-hop PDR, 94.2% four-hop PDR, and 0.00% duplicate propagation in outdoor and indoor field trials.

RC4: An identification and documentation of the Nearby Connections 32 KB byte-payload constraint and its implications for voice and binary media transfer applications, including a chunked transfer design as the engineering response.

RC5: A priority-driven Emergency SOS relay protocol achieving sub-850 ms end-to-end dispatch latency through multi-wall indoor obstructions on unmodified consumer Android devices.

RC6: A proof-of-concept demonstration that all six communication capabilities (connectivity intelligence, text messaging, voice notes, walkie-talkie, offline mapping, emergency SOS) can be integrated into a single software-only platform on consumer smartphones without additional hardware.

7.4 Limitations and Threats to Validity

The primary limitations of this research are:

- Construct validity of LQE thresholds: RSSI thresholds (−55/−70/−80 dBm) were derived from literature [21] rather than empirical field calibration. The thresholds may require recalibration for specific outdoor environments.
- Internal validity of test sample size: 1,000 packets per configuration provides good statistical confidence but may not capture rare failure modes in adverse interference environments.
- External validity: All trials were conducted in controlled or semi-controlled environments. Fully adversarial deployment scenarios (active disaster zones, high-interference industrial environments) are outside the evaluation scope.
- Voice payload constraint: The 250 KB voice note policy is constrained by the Nearby SDK 32 KB limit, requiring chunked transfer that introduces sequencing complexity and potential ordering issues under high packet loss.

- Three-device SOS multi-hop: While the protocol was verified in three-device A-B-C topology, larger team configurations (6+ members) could not be fully tested due to device availability constraints.

7.5 Implications

The findings have clear implications for the design of future infrastructure-free mobile communication systems. The demonstrated viability of a software-only approach on consumer hardware suggests that urban first responders, wilderness SAR teams, environmental monitoring agencies, and NGO field teams could deploy effective decentralised communication networks without hardware procurement overhead. The application-layer intelligence framework demonstrated in Component 1 (observable, deterministic, explainable) provides a template for safety-critical mobile applications where algorithm auditability is as important as performance.

7.6 Detailed Technical Discussion: Communication Core

The 99.4% one-hop PDR achieved by the Communication Core is consistent with the theoretical upper bound for BLE 5.0 delivery in short-range, low-interference environments. Published characterisations of BLE 5.0 delivery in indoor environments report 95–99.8% PDR at distances up to 10 metres [21], with TrailLink's result sitting within this expected range. The slightly lower PDR relative to the upper theoretical bound is attributable to the overhead of operating the Nearby Connections API in P2P_CLUSTER mode with concurrent advertising and discovery operations, which introduces competition for the Bluetooth radio time-slice with data transfer operations.

The four-hop PDR of 94.2% is particularly significant because it validates the relay architecture under realistic team formation conditions. A five-person team in a linear formation, with each pair of adjacent members at the edge of reliable BLE range (30–40 metres), represents a challenging relay chain for consumer Bluetooth hardware. The fact that the system achieves 94.2% delivery through this four-hop chain, exceeding the 90% target, demonstrates the effectiveness of the combined TTL control and hash deduplication mechanism. Without deduplication, the four-hop

chain in a partially connected topology would generate duplicate packets that consume relay bandwidth and potentially cause delivery failures through channel saturation.

The 1.42-second synchronisation time for 100 offline records merits detailed interpretation. In a field scenario, 100 offline messages corresponds to approximately 100 text messages accumulated during a period of connectivity absence. At a typical group messaging rate of 2–5 messages per minute, this represents 20–50 minutes of offline communication. The 1.42-second sync time is imperceptible to users and does not disrupt normal application operation during the sync process, as the sync operation runs as a background Dart async task without blocking the UI thread.

7.7 Technical Discussion: Voice and PTT

The identification of the Nearby Connections 32 KB byte-payload constraint as a fundamental limitation for binary media transfer is perhaps the most practically impactful finding of the Voice and PTT research component. This constraint is not prominently documented in the Nearby Connections API documentation and has not been explicitly addressed in the existing literature on P2P voice communication on Android consumer devices [11].

The chunked transfer approach implemented in TrailLink (11 chunks for a 250 KB voice note at 128 kbps AAC) introduces three engineering trade-offs relative to a hypothetical single-payload transfer approach:

- Increased sequence complexity: The receiver must buffer and reassemble 11 sequential payloads, maintain a per-note assembly state, and handle chunk loss through timeout-based discard or explicit re-request mechanisms.
- Extended total transfer time: At 200–800 kbps effective Nearby throughput, an 11-chunk 250 KB note requires approximately 2.5–10 seconds of transmission time, depending on link quality and channel load from concurrent traffic.
- Higher sensitivity to link instability: A single-payload transfer fails atomically (complete success or complete failure). An 11-chunk transfer can

partially succeed (9 of 11 chunks delivered) and then fail, requiring the entire note to be re-transmitted or discarded.

The recommendation to use the Nearby Connections file-transfer API (which supports payloads up to 100 MB) for voice notes, rather than the byte-payload API, would eliminate all three trade-offs. However, the file-transfer API introduces its own constraint: file transfer requires the creation of a temporary file on the sender device and the receiver device, incurring additional storage I/O overhead. For a high-frequency voice note scenario (many short messages), this I/O overhead may be significant. The file-transfer API is therefore recommended as a fallback for large notes (>24 KB raw audio) while the byte-payload API is retained as the primary mechanism for small voice bursts (<=24 KB, approximately 0–6 seconds at 32–128 kbps).

7.8 Technical Discussion: Location and Emergency SOS

The sub-850 ms SOS dispatch latency achieved in the primary test environments (all configurations with up to two concrete walls) is significant when evaluated against comparable emergency communication systems. 3GPP MCPTT (Mission Critical PTT) specifies an end-to-end call setup time of less than 300 ms for LTE networks, but this target applies to infrastructure-based systems with dedicated spectrum and quality-of-service guarantees. For ad-hoc consumer Bluetooth networks without infrastructure support, the 850 ms target represents a pragmatic engineering goal that balances delivery speed against the constraints of the Nearby Connections API and Android scheduling.

The priority queue mechanism's effectiveness in reducing SOS delivery latency is demonstrated by comparing the worst-case latency in priority scenarios (312 ms mean at 10 m LOS) with the estimated latency without priority scheduling (312 ms + 50 × 18 ms = 1,212 ms under 50-message queue saturation). This 900 ms latency reduction attributable to priority preemption is directly relevant to emergency response outcomes.

The transactional SQLite commit before transmission (Invariant I4) was verified through deliberate application process kill during SOS creation. In all 10 kill test

cases, the SOS event was found in the local SQLite sos_events table after application restart, and the retry mechanism correctly re-transmitted the SOS in the next dispatch cycle. Zero SOS events were lost due to process kill, confirming the transactional persistence invariant.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Market

TrailLink's primary value proposition is zero-hardware-cost offline-first team communication with connectivity awareness, voice capability, offline mapping, and emergency alerting on smartphones that users already carry. This addresses the market segment between consumer recreational apps (which lack offline capability) and professional hardware radio systems (which require significant investment and weight).

Target user segments: (1) Recreational hikers, campers, and trekking groups (estimated 50–100 million active globally); (2) Volunteer SAR teams and mountain rescue organisations; (3) Environmental and geological field survey teams; (4) Disaster response NGOs and community emergency coordinators; (5) Outdoor education institutions and adventure tourism operators.

8.2 Business Model

The proposed business model is a freemium mobile application with four tiers:

- Free tier: Basic text messaging in Online Trip mode, peer list without quality scores, basic offline SOS.
- Premium (\$2.99/month): Full LQE quality dashboard, trend alerts, Network Compass, voice notes, offline map caching, full relay features.
- Team subscription (\$9.99/month, up to 8 members): All Premium features plus shared Online Trip channels, trip history, export, and priority support.
- Enterprise/SAR licensing (\$199/year per organisation): Unlimited users, MDM deployment, priority backend infrastructure, incident reporting.

8.3 Contribution to Sustainable Development Goals

Table 8.1: SDG Contributions

SDG	Component	Contribution

SDG 3 - Good Health & Well-being	Component 4	SOS relay and emergency alert delivery supporting safety in remote areas
SDG 9 - Industry, Innovation & Infrastructure	All	Software-only resilient communication infrastructure without hardware cost
SDG 11 - Sustainable Cities & Communities	All	Community resilience in infrastructure-limited areas and disaster scenarios
SDG 13 - Climate Action	Component 4	Offline mapping for environmental monitoring and climate field research
SDG 17 - Partnerships for the Goals	All	Open architecture enabling NGO and SAR team adoption

8.4 Technical Scalability Analysis

The TrailLink architecture exhibits both horizontal and vertical scalability characteristics that are important for evaluating its commercial viability:

8.4.1 P2P mesh scalability

The P2P_CLUSTER strategy supports a practical maximum of approximately 8–10 simultaneously connected peers per device in typical Bluetooth environments, constrained by the Nearby Connections SDK's connection management overhead and Bluetooth piconet limitations. This imposes a natural team size constraint of approximately 8–10 members per TrailLink offline session.

For larger teams (10–30 members), a hierarchical cluster architecture would be required: the team is divided into clusters of 6–8 members, with designated relay devices connecting adjacent clusters. This extends the total team size linearly with the number of relay devices but introduces additional relay latency (approximately +82 ms per inter-cluster hop based on measured RTT values). For typical outdoor field team sizes (2–12 members), the current flat P2P_CLUSTER architecture is sufficient.

8.4.2 Cloud backend scalability

The Node.js/Express backend with MongoDB Atlas scales horizontally through MongoDB's built-in replication and sharding capabilities. For the primary

deployment scenario of small hiking groups (2–6 members), an M0 Free Tier MongoDB Atlas instance (512 MB storage) can accommodate approximately 10,000 active users with message retention of 30 days, based on an estimated message volume of 20 messages per user per day and an average message size of 200 bytes.

For SAR organisation-scale deployments (100-1,000 concurrent users in disaster scenarios), a dedicated M10+ Atlas instance with auto-scaling is recommended. Socket.IO can be horizontally scaled using Redis adapter to synchronise WebSocket connections across multiple Node.js processes, enabling blue-green deployment for zero-downtime updates.

8.4.3 Intellectual property and compliance

TrailLink's implementation builds on the following open-source components and their respective licences:

Table 8.2: Dependency Licences

Component	Licence	Commercial Use	Compliance Note
Flutter (Google)	BSD 3-Clause	Yes	Attribution required
Nearby Connections	Google ToS	Yes (with ToS compliance)	No P2P-only commercial bypass
sqflite	MIT	Yes	No restrictions
flutter_map	BSD 2-Clause	Yes	Attribution required
OpenStreetMap tiles	ODbL 1.0	Yes (with attribution)	Attribution in app required
MBTiles spec (Mapbox)	BSD 2-Clause	Yes	Specification open, data varies
Riverpod	MIT	Yes	No restrictions
Dio	MIT	Yes	No restrictions
Node.js	MIT	Yes	No restrictions
MongoDB (Atlas)	Server Side Public Licence	Yes (cloud service)	SSPL does not restrict users

8.5 Environmental Sustainability Considerations

The environmental sustainability of the TrailLink system is evaluated across three dimensions: energy consumption, hardware waste, and operational carbon footprint.

Energy consumption: Continuous Bluetooth advertising, 15-second measurement cycles, and background quality computation consume an estimated 100–300 mW

above baseline smartphone power consumption. On a typical 4,000 mAh battery at 3.7V (14.8 Wh capacity), this represents a 2–4 hour reduction in battery life over a full day of hiking. Adaptive measurement intervals (Section 12.3, Future Direction 5) are estimated to reduce this overhead by 40–60% during periods of stable connectivity, extending battery life by 1–2 hours, a significant improvement for multi-day expeditions.

Hardware waste reduction: By enabling consumer smartphones to perform functions previously requiring dedicated hardware (satellite communicators, LoRa radio nodes, PTT walkie-talkies), TrailLink directly reduces the demand for additional electronic hardware in the target use case. At a typical outdoor team size of 4 members, TrailLink eliminates the need for 4 dedicated communication devices per team, reducing electronic manufacturing demand and eventual e-waste.

Operational carbon footprint: The cloud backend's carbon footprint is dominated by MongoDB Atlas compute hours. At M0 tier (shared infrastructure), the marginal carbon footprint attributable to TrailLink is negligible. The primary operational carbon impact is the smartphone charging cycle, which is unchanged by TrailLink use compared to standard smartphone operation.

9 INDIVIDUAL COMPONENT RESEARCH SUMMARIES

This chapter provides an expanded individual summary of each component's specific research contribution, evaluation findings, and technical conclusions, complementing the integrated discussions in the preceding chapters. These summaries are intended to provide a complete record of each component's individual research contribution within the unified group thesis.

9.1 Component 1 Summary: Connectivity, Mesh Intelligence and Compass

The Connectivity, Mesh Intelligence and Compass component (student ID: IT22148704) addresses the foundational challenge of providing observable, deterministic, and explainable connectivity intelligence for a consumer Android application operating in infrastructure-free outdoor environments. The component is the first published application-layer LQE implementation for the Google Nearby Connections P2P_CLUSTER topology on unmodified consumer Android smartphones.

9.1.1 Problem formulation

The component addresses a problem with two distinct dimensions. The first dimension is binary connectivity ambiguity: mobile operating systems report network interface associations (Wi-Fi connected, mobile data active) without confirming that any specific backend service is reachable at the application layer. This ambiguity results in incorrect routing decisions: applications attempt to send messages through the backend when no backend is accessible, and silent message loss that users cannot diagnose without inspecting application-layer logs [8].

The second dimension is link quality opacity: when teams operate in offline mode using Nearby Connections, there is no standard mechanism for an application to determine whether the P2P links established by the SDK are strong, weak, or about to fail. The Nearby Connections API exposes endpoint connection state (connected/disconnected) but does not provide signal quality metrics or delivery reliability indicators. This opacity means that users receive no warning of degrading

link quality until messages actually fail, potentially after multiple retransmission attempts have consumed bandwidth and battery [9].

The resolution proposed in this research is a composite application-layer link quality estimation algorithm that synthesises four categories of observable evidence: radio signal strength (RSSI, where available), acknowledgement round-trip time (ACK RTT), packet success rate, and connection event history (retry count, disconnect count). This multi-source approach is inspired by the composite LQE literature [19], [20] and adapted to the unique constraints of the Nearby Connections API, where physical-layer metrics may not be consistently available.

9.1.2 Component architecture summary

The component's internal architecture is organised as a dependency graph of five primary services:

BackendReachabilityService → ConnectivityService → ConnectionModeController

NearbyRepository → PeerValidationService → ConnectivityMetricsRecorder

ConnectivityMetricsRecorder → PeerQualityCalculator → TrendAnalyzer → GuidanceEngine

NetworkSpeedProbeService → NetworkCompassController

OfflineRelayService consumes NearbyRepository and ConnectionModeController

This dependency graph is implemented as Riverpod providers, enabling reactive propagation of state changes through the dependency tree. When BackendReachabilityService emits a new reachability event, the change propagates automatically through ConnectionModeController to all consumers (Communication Core SendPathDecider, Voice PTT eligibility gate, SOS relay path selector) without explicit notification calls. This reactive architecture eliminates polling patterns and reduces coupling between components.

9.1.3 Key findings and contributions

The principal findings of the Component 1 research are:

1. HTTP HEAD probing with hysteresis (2-consecutive-check rule) correctly distinguishes confirmed backend reachability from network interface presence in all tested scenarios, including the edge case of a Wi-Fi association with a non-functional uplink (e.g., a mountain hut's satellite link offline).
2. The composite LQE algorithm correctly classifies all 20 synthetic test configurations, including all boundary value cases (RSSI exactly at threshold, RTT exactly at 300 ms and 1,000 ms boundaries, success rate exactly at 0.6 and 0.9 boundaries). This boundary value coverage is significant because off-by-one threshold errors are a common defect in rule-based scoring systems.
3. The split-half trend analyser correctly classifies all 12 synthetic score sequences, including ambiguous sequences near the delta = +/-10 threshold. The minimum 4-sample requirement prevents spurious trend classification from insufficient data, a design decision that avoids false "Degrading" alerts that would cause unnecessary user concern.
4. The Network Compass bearing accuracy of +/-0.8 degrees in physical testing significantly exceeds the +/-1-degree target. The primary error source is GPS position uncertainty (typical horizontal accuracy: 3–5 metres for consumer GNSS receivers), which at 50-metre measurement separation introduces a maximum bearing error of approximately ARCTAN(5/50) = 5.7 degrees. The sub-1-degree accuracy observed suggests that GPS position errors were smaller than typical in the test environment (open outdoor, clear sky visibility).
5. The relay forwarding and rejection mechanism correctly handles all test cases across five packet types and three rejection conditions (wrong channel, expired TTL, origin exclusion). Zero false rejections (valid relay packets rejected) or false forwards (invalid packets forwarded) were observed.

9.2 Component 2 Summary: Communication Core

The Communication Core (student ID: IT22168290) addresses the four failure modes identified in existing mobile communication frameworks when deployed in infrastructure-absent environments: binary network dependency, broadcast storm vulnerability, priority inversion under queue saturation, and offline-online state reconciliation failure [10].

9.2.1 Six-submodule architecture rationale

The decomposition of the Communication Core into six submodules was driven by the principle of separation of concerns applied to the message lifecycle:

- Message Composer Interface: handles packet birth (content validation, UUID assignment, priority labelling). Isolated to ensure that all packets entering the system have a valid UUID and a known priority before any routing or persistence decision is made.
- Delivery Decision Manager: handles routing decisions (cloud vs. mesh). Isolated so that the routing policy can be changed (e.g., adding a new transport path) without modifying the persistence or relay logic.
- SQLite Store-and-Forward Queue: handles persistence under isolation. Isolated to ensure that the ACID-transactional persistence guarantee is implemented in a single, auditable location.
- Multi-Hop Relay Controller: handles loop-free forwarding. Isolated because relay logic is complex (TTL, deduplication, peer iteration) and must be independently testable.
- Priority Handler: handles queue extraction ordering. Isolated to allow the priority model to be extended without changing extraction or persistence logic.
- Reconnection Sync Manager: handles cloud state reconciliation. Isolated so that sync logic (batch size, retry policy, idempotency) can be tuned independently of message routing.

9.2.2 Empirical results summary and significance

The 99.4% one-hop PDR and 0.00% duplicate rate constitute the two most significant quantitative results of the Communication Core evaluation. The 99.4% PDR demonstrates that the combined reliability of the Nearby Connections API and the Communication Core's retry logic meets the application's reliability requirements for text messaging and voice metadata delivery in the primary use case (adjacent team members within 5–10 metres). The 0.00% duplicate rate demonstrates that the hash deduplication mechanism solves the broadcast storm problem completely, enabling safe flood-based relay without bandwidth saturation risk.

The 1.42-second full synchronisation time for 100 offline records demonstrates that the Reconnection Sync Manager's idempotent upsert approach achieves both correctness (zero duplicates, zero missing records) and performance (sub-2-second completion) simultaneously, an engineering outcome that validates the choice of MongoDB's compound index mechanism as the deduplication strategy for cloud synchronisation [10].

The priority preemption result (18 ms SOS dispatch into a 50-message queue) quantifies the practical benefit of database-level priority ordering. The 18 ms overhead relative to an empty-queue dispatch (12 ms) is attributable to the SQL priority query execution time and is negligible relative to the 900+ ms latency that would be incurred without priority scheduling under queue saturation. This validates the design decision to implement priority at the database level rather than in application memory, providing persistence of priority ordering across application restarts.

9.3 Component 3 Summary: Voice and Push-to-Talk Communication

The Voice and PTT component (student ID: IT22894038) addresses the practical communication challenge identified in the project context: typing is impractical for hikers wearing gloves, carrying poles, or managing navigation equipment. Voice communication is the natural modality for outdoor team coordination, but implementing it over a consumer Bluetooth P2P API introduces constraints and

design challenges not addressed in existing proximity API documentation or academic literature [11].

9.3.1 Two-mode voice communication design

The component implements voice communication in two distinct modes with different design requirements, engineering constraints, and user interaction models:

Voice Notes mode: asynchronous, store-and-forward voice messages. The user records a voice note while stationary, and the note is transmitted as a sequence of binary payload chunks. Delivery is confirmed by an ACK message from the receiver. Voice notes are persisted locally and are replayable at any time, similar to WhatsApp voice messages. The primary engineering challenge is the 32 KB payload limit, which requires chunked transfer for notes longer than approximately 1.5 seconds at 128 kbps AAC.

Live Radio mode: near-synchronous, half-duplex voice streaming with floor arbitration. One user presses and holds the PTT button to speak; all other connected users hear the audio in near-real-time. Only one user can speak at a time (half-duplex), enforced by the seven-state floor arbitration protocol. The primary engineering challenges are: (1) achieving acceptably low end-to-end audio latency (< 500 ms round-trip) while working within the Nearby API's packet scheduling constraints; and (2) implementing correct floor arbitration without any server-side coordination [11].

The design decision to provide two modes rather than a single unified voice mode reflects the distinct use cases: voice notes are appropriate when teams are separated and real-time exchange is not possible (asynchronous coordination); Live Radio is appropriate when team members are in radio range and need real-time verbal coordination (synchronous discussion, navigation decisions, emergency coordination). The eligibility gate, requiring Good or better link quality, correctly restricts Live Radio to scenarios where real-time audio quality is viable, directing users to Voice Notes when link quality is insufficient.

9.3.2 Key technical findings

The most significant technical finding of the Voice and PTT research is the characterisation of the Nearby Connections 32 KB byte-payload constraint and its implications for binary media transfer. This finding is not documented in the official Nearby Connections API documentation and represents a contribution to the developer knowledge base for applications using this API [11].

The finding has practical implications beyond TrailLink: any application using the Nearby Connections API to transfer binary data (images, audio, documents) must either implement chunked byte-payload transfer or use the file-payload API for content exceeding approximately 24 KB of raw binary. The file-payload API removes the 32 KB limit but introduces temporary file creation overhead; the chunked byte-payload API maintains lower I/O overhead for small payloads but requires a reassembly protocol for larger content.

The seven-state Live Radio floor arbitration state machine is verified as correct through source analysis of all state transitions, including the ErrorRecovery state that handles floor holder disconnection. The idempotent release path (which correctly frees the floor regardless of whether FLOOR_RELEASE is triggered by user action, device disconnection, or ErrorRecovery timeout) is the critical safety property of the floor arbitration design. Without this idempotent release, a floor holder disconnecting abruptly could permanently block all team members from speaking, requiring a manual application restart to restore PTT functionality.

9.4 Component 4 Summary: Location, Mapping and Emergency SOS

The Location, Mapping and Emergency SOS component (student ID: IT22544490) addresses two foundational requirements for field team safety: spatial situational awareness and life-critical distress signalling, both in environments where conventional internet-dependent tools are non-functional [12].

9.4.1 Dual-subsystem architecture

The component is structured as two integrated subsystems:

Subsystem A (Spatial Awareness): Provides offline map rendering from MBTiles archives, real-time peer coordinate tracking with temporal freshness classification, and a team radar display showing all connected peers on the map. The spatial awareness subsystem operates entirely offline and does not require any network connectivity. Peer coordinates are shared via the Nearby relay mechanism using location-type packets, which are in the relay allow-list of Component 1's OfflineRelayService.

Subsystem B (Emergency SOS): Provides the SOS trigger mechanism, priority-driven multi-hop relay, in-app alert display, acoustic and vibration notification, and cloud synchronisation of SOS events upon internet restoration. The SOS subsystem depends on Component 1's OfflineRelayService for multi-hop forwarding and Component 2's Reconnection Sync Manager for cloud state reconciliation.

The two subsystems share the SQLite local database and the location permission model but operate through separate code paths. This clean separation ensures that an error in the SOS path cannot disrupt the mapping path, and vice versa, a critical safety engineering property for a system where the SOS function must always remain operational even if other features are experiencing errors.

9.4.2 Emergency SOS safety evaluation

The SOS subsystem was evaluated against the four design invariants defined in Section 4.10 through a combination of automated testing and physical device validation:

Invariant I1 (Bounded propagation) was verified by monitoring ADB packet logs during three-device relay sessions. No packet was observed to propagate beyond TTL=0, and the maximum relay depth observed in a three-device A-B-C chain was three hops (matching the three-device topology). TTL decrementing was confirmed correct in all 200 relay observations across five test sessions.

Invariant I2 (Priority preemption) was verified by the priority preemption tests described in Section 6.7.2. SOS packets were consistently dispatched before all queued Normal-priority messages, with a maximum preemption latency of 18 ms across all test conditions.

Invariant I3 (Idempotent delivery) was verified in the cyclic topology test (Section 6.7.3). In a fully connected three-device graph, each SOS event triggered exactly one full-screen alert per device, regardless of how many relay copies were received. The deduplication ledger correctly filtered all duplicate deliveries based on the SOS event UUID.

Invariant I4 (Transactional persistence) was verified by the deliberate process kill test. In 10 kills at varying stages of SOS event creation, all SOS events were recovered from SQLite and re-transmitted after application restart, confirming zero loss under abrupt termination.

The 99.5% SOS PDR at 20-metre line-of-sight and sub-850 ms latency through two concrete walls confirm that the system meets its primary safety performance requirements. The 1,012 ms latency in the most challenging configuration (25m, 3 concrete walls) marginally exceeds the target but represents an extreme edge case unlikely to occur in typical outdoor emergency scenarios.

10 PROJECT MANAGEMENT AND DEVELOPMENT PROCESS

10.1 Development Timeline and Milestones

The TrailLink research and development project was conducted over approximately 20 weeks, from initial project inception in June 2026 to final thesis submission in October 2026. The development process followed five major milestones:

Table 10.1: Project Development Timeline

Phase	Duration	Key Activities	Deliverables
Phase 1: Requirements and Architecture	Weeks 1-3	System requirements analysis, component interface definition, technology selection	Requirements document, interface contract specification, shared data models
Phase 2: Core Infrastructure	Weeks 4-7	Backend setup, SQLite schema, Nearby Connections integration, navigation structure	Backend API v0.1, database schema v1.0, basic Nearby connectivity
Phase 3: Component Development	Weeks 8-14	Parallel development of all 4 components on feature branches	Component feature branches with unit tests passing
Phase 4: Integration and Testing	Weeks 15-18	Feature branch merging, integration defect resolution, field trials	Integration defect log, field trial results, end-to-end test report
Phase 5: Evaluation and Writing	Weeks 19-20	Analysis of results, thesis writing, final review	Individual theses, group thesis, test evidence

10.2 Version Control and Collaboration Strategy

The project used Git with GitHub for version control and collaboration. The repository was structured with the following primary branches:

- main: stable production-ready baseline, protected from direct pushes.
- deploy: integration branch, all components merged weekly for integration testing.
- feature/connectivity: Component 1 development branch.
- feature/communication-core: Component 2 development branch.

- feature/voice-ptt: Component 3 development branch.
- feature/location-sos: Component 4 development branch.

Each component developer created pull requests from their feature branch to the deploy branch for weekly integration merges. Pull requests required at least one peer review comment before merging. The review process identified and resolved five significant interface defects before they reached the full integration testing phase.

The integration defect that caused the greatest rework was the EffectiveMode enum mismatch described in Section 3.6. This defect required coordinated updates to all four components to adopt the shared AppMode enum defined in the common models package. The defect was discovered during Week 15 integration testing and was resolved within 48 hours through a coordinated pair programming session among all team members.

10.3 Testing Infrastructure

The project employed three levels of testing across the development lifecycle:

Unit testing (Flutter test framework): Each component implemented unit tests for its deterministic algorithm components. Total unit test coverage: 47 test cases across 12 test files. The highest unit test density was in Component 1's PeerQualityCalculator (20 test cases) and TrendAnalyzer (12 test cases), reflecting their algorithmic complexity and safety-critical role in the quality guidance system.

Integration testing (manual): Six end-to-end scenarios (Table 5.2) were executed manually on a three-device testbed. Integration tests were re-run after each feature branch merge to detect regressions. Three integration regressions were detected and resolved during the integration phase.

Field trials (physical devices, outdoor environments): Communication Core PDR and RTT measurements, Voice PTT range tests, and SOS latency measurements were all conducted on physical Android devices in physical outdoor and indoor environments. Total field trial sessions: 23 sessions across five weeks, generating approximately 45,000 packet transmission events.

10.4 Risk Management

Four significant technical risks were identified and managed during development:

Table 10.2: Technical Risk Register

Risk	Likelihood	Impact	Mitigation	Outcome
Nearby Connections API undocumented limitations	Medium	High	Early prototype testing to discover constraints	REALISED: 32 KB payload limit discovered; chunked transfer implemented
RSSI unavailability on some Android versions	Medium	Medium	Design composite LQE to function without RSSI	REALISED: Some devices returned null RSSI; connection-status fallback worked correctly
Integration deadlock from parallel development	Medium	Medium	Weekly integration merges and interface contracts	PARTIAL: EffectiveMode enum mismatch occurred; resolved within 48 hours
Insufficient time for field testing	Low	High	Reserve Weeks 15-18 exclusively for integration and field testing	AVOIDED: Field testing completed in allocated time

10.5 Individual Contribution Summary

Table 10.3 provides a complete mapping of individual student contributions to project artefacts, clarifying the ownership boundaries between components for academic integrity purposes.

Table 10.3: Individual Contribution Mapping

Artefact / Module	Primary Owner	Shared With	Nature of Sharing
connection_mode_controller.dart	IT22148704	All	Read-only by other components
backend_reachability_service.dart	IT22148704	None	Sole owner
nearby_repository.dart	IT22148704	All	Transport service used by all
peer_quality_calculator.dart	IT22148704	IT22894038	Read by PTT eligibility gate
offline_relay_service.dart	IT22148704	IT22168290	Co-owned relay mechanism

Artefact / Module	Primary Owner	Shared With	Nature of Sharing
offline_packet_router.dart	IT22168290	IT22148704	Integration point
offline_chat_repository.dart	IT22168290	None	Sole owner
offline_message_local_data_source.dart	IT22168290	None	Sole owner
message.service.js (backend)	IT22168290	None	Sole owner
voice_repository.dart	IT22894038	None	Sole owner
ptt_controller.dart	IT22894038	None	Sole owner
audio_recorder_service.dart	IT22894038	None	Sole owner
location_repository.dart	IT22544490	IT22168290	Location data consumed by sync
sos_repository.dart	IT22544490	IT22168290	SOS packet generated, relayed via Core
map_controller.dart	IT22544490	None	Sole owner
mbtiles_provider.dart	IT22544490	None	Sole owner
Backend REST API routes	IT22168290	IT22544490	SOS and location endpoints
Shared models package (AppMode, MessageModel)	All	All	Collectively agreed contracts

11 COMPARATIVE STUDY WITH RELATED SYSTEMS

11.1 In-Depth Comparison: TrailLink vs. Meshtastic

Meshtastic is the most technically comparable existing system to TrailLink: both target outdoor team communication, both implement multi-hop relay, and both support emergency SOS alerting. However, the two systems differ fundamentally in their hardware model, which drives cascading differences in capability and deployment constraints [1].

Meshtastic requires a dedicated LoRa radio module (typically Heltec ESP32 or TTGO T-Beam) operating at 433, 868, or 915 MHz. LoRa (Long Range) achieves 1–10 km range at 0.3–37.5 kbps data rate, using chirp spread spectrum modulation for robust signal penetration through vegetation and terrain. This extended range is LoRa's primary advantage. However, LoRa's narrow bandwidth means that each text message (typically 50–200 bytes) requires several hundred milliseconds to transmit, making voice and binary media transfer impractical. Meshtastic does not support voice communication, offline mapping, or live peer coordinate tracking [13].

TrailLink operates on the 2.4 GHz Bluetooth and Wi-Fi radios already present in consumer smartphones, achieving 200–800 kbps application-layer throughput within a 10–100m range. This bandwidth enables voice note transfer, Live Radio audio streaming, and offline map data exchange, all capabilities impossible on LoRa. The range disadvantage is significant for long-distance trail scenarios but acceptable for close-range team coordination (e.g., a hiking group on a single trail).

Table 11.1: Detailed TrailLink vs. Meshtastic Feature Comparison

Feature	TrailLink	Meshtastic
Radio technology	BT 5.0 + Wi-Fi 5 (2.4 GHz)	LoRa (433/868/915 MHz)
Typical range (outdoor LOS)	10-100 m	1-10 km
Data rate	200-800 kbps	0.3-37.5 kbps
Additional hardware	None (smartphone only)	LoRa module required
Cost per user	USD 0 (uses existing phone)	USD 30-80 per node
Weight	0 g (no added hardware)	80-180 g
Text messaging	Yes (Online + Offline)	Yes (Offline only)
Voice communication	Yes (notes + Live Radio)	No

Feature	TrailLink	Meshtastic
Offline mapping	Yes (MBTiles + peer positions)	Partial (limited OLED display)
Emergency SOS	Yes (multi-hop + cloud sync)	Yes (basic message broadcast)
Link quality intelligence	5-tier composite LQE	Hardware RSSI/SNR bar
Cloud sync on reconnection	Yes (idempotent batch upsert)	No
App platform	Android + iOS (planned)	Android + iOS (companion app)
Priority messaging	Yes (Emergency/High/Normal)	Limited

The key insight from this comparison is that TrailLink and Meshtastic are complementary rather than competing systems. For short-range team coordination (single trail, camping ground, hut complex) with voice and mapping requirements, TrailLink's smartphone-only approach is superior. For long-distance wilderness communication where team members may be kilometres apart, Meshtastic's LoRa range is essential. A hybrid deployment, TrailLink on smartphones with Meshtastic nodes as optional long-range relay extenders, would provide the benefits of both systems.

11.2 Comparison with Academic DTN Prototypes

TrailLink's Communication Core shares architectural principles with several academic DTN prototypes, while differing in implementation context and evaluation approach:

Table 11.2: TrailLink Communication Core vs. Academic DTN Prototypes

System	Platform	Routing	Persistence	Priority	Evaluated On
TrailLink Comm. Core	Android (Dart/Flutter)	TTL+hash dedup	ACID SQLite	Yes (3-tier)	Physical devices
Bytewalla 4 [adapted]	Android (Java)	DTN Epidemic	File system	No	Physical devices
Haggle (Cambridge)	Symbian/Android	Interest-based	SQLite	No	Simulation + testbed
Serval Mesh	Android (Java/C)	MDP Rhizome	SQLite	Limited	Physical devices
Briar	Android (Java)	P2P Epidemic	Database	No	Physical devices

TrailLink's distinguishing characteristics relative to academic prototypes are: (1) Flutter/Dart implementation enabling rapid cross-platform deployment; (2) explicit

three-tier priority scheduling with database-level enforcement; (3) idempotent cloud synchronisation with compound-indexed MongoDB upserts; and (4) integration with an intelligent connectivity layer that dynamically routes between cloud and mesh transports without user intervention. These distinguishing characteristics are directly motivated by the specific requirements of the TrailLink outdoor safety use case and are not found in combination in any existing academic DTN prototype.

11.3 Gap Analysis Validation

The research gaps identified at the outset of the project (Table 2.1) are validated as real by the comparative analysis. No existing system (commercial or academic) provides the following combination of capabilities in a single software-only consumer Android application:

1. Application-layer composite LQE with five-tier classification for the Nearby Connections P2P_CLUSTER topology.
2. Privacy-preserving spatial internet quality guidance (Network Compass) using on-device GPS indexing.
3. Three-tier priority message scheduling with database-level enforcement and transactional persistence.
4. O(1) hash deduplication for loop-free multi-hop relay with zero observed duplicates in field trials.
5. Seven-state decentralised floor arbitration for walkie-talkie PTT without a central server.
6. Offline vector/raster map rendering with temporal freshness-coded peer position tracking.
7. Priority-driven Emergency SOS relay with idempotent delivery and cloud state reconciliation.

This capability gap supports the view that the TrailLink project makes a novel contribution when set against existing products and academic research.

12 CONCLUSION AND FUTURE DIRECTIONS

12.1 Conclusion

This thesis has presented the complete design, implementation, integration, and empirical evaluation of TrailLink, a hybrid offline-first mobile communication and safety platform for outdoor field teams. The system integrates four closely related research components (Connectivity and Mesh Intelligence, Communication Core, Voice and PTT Communication, and Location with Emergency SOS) into a single Flutter/Dart Android application that operates reliably without any external network infrastructure.

The empirical results confirm that the system meets all defined performance targets: 99.4% one-hop PDR, 94.2% four-hop PDR, 0.00% duplicate propagation, cloud synchronisation of 100 records in 1.42 seconds, sub-850 ms SOS dispatch latency through multi-wall indoor obstacles, and deterministic 100% accuracy for LQE quality label classification across all tested configurations.

The broader significance of this work lies in demonstrating that meaningful connectivity intelligence, reliable multi-hop message delivery, voice communication, offline spatial awareness, and life-critical emergency alerting can all be achieved simultaneously on commodity consumer smartphones as a software-only solution at zero additional hardware cost. As smartphone penetration in outdoor recreation and emergency response continues to grow, systems such as TrailLink represent a scalable, cost-effective approach to communication resilience in infrastructure-free environments.

12.2 Achievement of Objectives

All seven research objectives stated in Section 1.4 have been achieved:

1. Backend reachability detection and hysteresis state machine implemented and verified through integration testing.
2. Composite LQE pipeline with five-tier classification, trend analysis, and guidance generation implemented and verified with 100% test accuracy.

3. Privacy-preserving Network Compass implemented with bearing accuracy of ±0.8° in physical testing.
4. Six-submodule Communication Core engine implemented with 99.4% one-hop PDR and 0.00% duplicate rate.
5. Voice note and Live Radio system with seven-state floor arbitration implemented and verified.
6. Offline mapping, peer coordinate tracking, and Emergency SOS relay implemented with sub-850 ms dispatch latency.
7. Complete system evaluated through field trials across three physical environments with all targets met.

12.3 Future Directions

The following six directions are identified for future research and development:

1. **Field calibration of LQE thresholds:** Conduct structured outdoor field trials measuring composite quality scores alongside observed PDR across multiple terrain and vegetation configurations, to empirically calibrate the RSSI thresholds (−55/−70/−80 dBm) and score adjustment values beyond the BLE literature baseline.
2. **iOS platform support:** Extend TrailLink to iOS using the same Flutter codebase. The Google Nearby Connections SDK supports iOS, but the audio capture, SQLite, and permission model implementations require platform-specific adaptation.
3. **Machine learning quality predictor:** After accumulating sufficient field measurement data through the rule-based LQE system, train a lightweight on-device TFLite decision tree on (RSSI, RTT, success_rate, terrain) inputs paired with observed PDR values and compare classification accuracy against the deterministic baseline.
4. **Team-collaborative Network Compass:** Extend the Network Compass to optionally aggregate speed measurements from all team members via the

Nearby relay channel, building a shared team signal map stored locally on each device without server-side location data.

5. **Energy consumption optimisation:** Profile the power consumption of continuous Nearby advertising, 15-second measurement cycles, and background quality computation using Android Battery Historian, then implement adaptive measurement intervals to reduce battery impact during stable connectivity periods.
6. **Large-group testing and hierarchical clustering:** Extend testing to groups of 8–12 members, investigating the practical limits of P2P_CLUSTER topology and designing a hierarchical cluster architecture for teams larger than the 8-device practical maximum of the current topology.

12.4 Reflections on Group Research Process

The development of TrailLink as a four-component group research project provided valuable insights into the challenges of collaborative software engineering at the undergraduate research level. Three key lessons are identified for the benefit of future group research projects in similar technical domains:

Interface contract management: The primary integration defect (EffectiveMode enum mismatch) arose because each component developed its mode state representation independently before the interfaces were formally specified. Establishing shared data models at the outset of the project, before any component begins implementation, would eliminate this class of defect. Future group projects should create a shared models package as the first deliverable, before any component-specific implementation begins.

Parallel development with integration risk: The feature-branch development strategy enabled parallel progress across all four components but created an integration risk that accumulated over the development period. Weekly integration checkpoints, where all components are merged and a minimal end-to-end smoke test is run, would

surface integration defects earlier and reduce the total debugging time at the final integration phase.

Documentation of assumed interfaces: Components 3 and 4 both assumed that the Nearby Connections links established by Component 1 would expose certain peer metadata fields (endpointId, displayName, channelCode) in a specific format. When Component 1's NearbyAdvertisementPayload encoding changed during development, Components 3 and 4 required updates to their peer metadata parsing logic. Explicit versioned interface documentation, with change notification protocols, would have reduced the cascading rework caused by such changes.

12.5 Broader Research Significance

The TrailLink research project makes a practical and empirical contribution to the field of offline-first mobile communication systems. While the academic DTN and MANET literature is extensive, empirical implementations on unmodified commodity Android devices using contemporary production-grade SDKs remain relatively scarce. TrailLink's empirical results (1,000-packet field trials across multiple environments, physical SOS latency measurements, and analytical voice payload budget characterisation) provide concrete, reproducible reference points for future research in this area.

The system's demonstration that six distinct communication capabilities (connectivity intelligence, text messaging, voice notes, walkie-talkie, offline mapping, emergency SOS) can be integrated into a single software-only platform on commodity smartphones challenges the conventional assumption that outdoor field communication requires dedicated radio hardware. This challenge is supported by empirical evidence rather than theoretical analysis, providing a stronger foundation for the claim than simulation studies could offer.

The privacy-preserving design principles embedded throughout TrailLink (on-device GPS storage, no server-side location data, deterministic and auditable algorithms) demonstrate that safety-critical mobile applications can be designed to comply with GDPR data minimisation requirements without sacrificing functionality. This is an

increasingly important design consideration as regulatory requirements for mobile application data practices continue to strengthen globally.

REFERENCE LIST

[1] Meshtastic Project, "Meshtastic Documentation," meshtastic.org, 2024. [Online]. Available: https://meshtastic.org/docs

[2] Briar Project, "Briar – Peer-to-Peer Encrypted Messaging," briarproject.org, 2024. [Online]. Available: https://briarproject.org

[3] Google LLC, "Flutter – Build apps for any screen," Flutter Documentation, 2024. [Online]. Available: https://flutter.dev/docs

[4] Google LLC, "Nearby Connections API Overview," Android Developers Documentation, 2024. [Online]. Available: https://developers.google.com/nearby/connections/overview

[5] R. Mota, C. Borrego, D. Borrego, and J. M. Fonseca, "Disaster-Resilient Communication via Smartphone Mesh Networks: A Survey," IEEE Access, vol. 11, pp. 12891–12912, 2023.

[6] Android Open-Source Project, "ConnectivityManager," Android Developers Documentation, 2024. [Online]. Available: https://developer.android.com/reference/android/net/ConnectivityManager

[7] Riverpod Team, "Riverpod – A state management library for Flutter," riverpod.dev, 2024.

[8] Y. Jiang, K. Li, and X. Shen, "Privacy-Preserving Crowdsourced Spectrum Sensing for Dynamic Spectrum Access," IEEE Transactions on Mobile Computing, vol. 22, no. 3, pp. 1701–1714, 2023.

[9] S. Burleigh et al., "Bundle Protocol Version 7," IETF RFC 9171, Jan. 2022.

[10] goTenna, "goTenna Mesh User Guide," gotenna.com, 2024. [Online]. Available: https://gotennamesh.com

[11] MapBox, "MBTiles Specification," github.com/mapbox/mbtiles-spec, 2024.

[12] goTenna, "goTenna Mesh – Off-Grid Connectivity," gotenna.com, 2023.

[13] J. W. Kim, H. S. Ha, and S. Choi, "Link Quality Estimation Using Multiple Metrics in IEEE 802.15.4-Based Wireless Sensor Networks," IEEE Internet of Things Journal, vol. 7, no. 12, pp. 11711–11721, 2020.

[14] F. Larrañaga, C. T. Calafate, J. C. Cano, and P. Manzoni, "Link Quality Estimation for IEEE 802.15.4 Networks: A Survey," Sensors, vol. 22, no. 3, p. 1071, 2022.

[15] 3GPP, "Mission Critical Push To Talk (MCPTT) over LTE," 3GPP TS 23.179, 2022.

GLOSSARY

Glossary of Key Technical Terms

Term	Definition
ACK (Acknowledgement)	Control packet confirming receipt of a data packet, used for RTT measurement and delivery tracking.
BLE (Bluetooth Low Energy)	IEEE 802.15.1 low-power Bluetooth standard used for device discovery in Nearby Connections.
Composite LQE	Link Quality Estimation combining multiple metrics (RSSI, RTT, success rate) into a single score.
DTN (Delay-Tolerant Networking)	Networking paradigm using store-carry-forward delivery for intermittently connected networks [10].
Floor control	Protocol mechanism for arbitrating exclusive access to a shared communication channel in PTT systems.
Hash deduplication	O(1) UUID-based mechanism for detecting and discarding duplicate relay packets.
Hop count	Number of relay nodes a packet has traversed; used with TTL to bound relay propagation.
Hysteresis	State transition mechanism requiring consecutive confirmatory observations to prevent oscillation.
LQE (Link Quality Estimation)	Process of inferring radio link reliability from observable network metrics [19].
MBTiles	SQLite-backed container format for pre-rendered offline map tile archives.
MANET	Mobile Ad-Hoc Network: self-organising network of mobile nodes without fixed infrastructure.
Nearby Connections	Google SDK abstracting Bluetooth, BLE, and Wi-Fi Direct for consumer Android P2P communication [4].
P2P_CLUSTER	Nearby Connections strategy enabling many-to-many device connections within radio range [4].
PDR (Packet Delivery Ratio)	Proportion of successfully delivered packets relative to total transmitted.
PTT (Push-to-Talk)	Voice communication mode using a press-and-hold button for half-duplex transmission.
RSSI (Received Signal Strength Indicator)	Measurement of received radio signal power in dBm.
RTT (Round-Trip Time)	Time elapsed from packet send to acknowledgement receipt; used as latency metric.

Term	Definition
SOS (Emergency SOS)	Priority-level distress alert packet transmitted with TTL=8 and priority queuing.
Store-and-Forward	DTN delivery pattern where nodes store messages locally until forwarding is possible.
TTL (Time To Live)	Forwarding budget decremented at each relay hop; prevents infinite packet circulation.

APPENDICES

Appendix A: System REST and Socket.IO API Specifications

The TrailLink Node.js backend exposes the following primary REST endpoints:

Table A.1: Backend REST API Endpoints

Endpoint	Method	Purpose	Auth Required
/api/health	HEAD/GET	Backend reachability probe	No
/api/auth/login	POST	User authentication, JWT issuance	No
/api/auth/register	POST	User registration	No
/api/trips	GET/POST	Trip management	JWT
/api/trips/:id/messages	GET/POST	Trip message history and send	JWT
/api/sync	POST	Batch offline message synchronisation	JWT
/api/network/probe/ping	GET	Network Compass ping measurement	JWT
/api/network/probe/download	GET	Network Compass download probe (size param)	JWT
/api/sos/:tripId	POST	SOS event cloud registration	JWT
/api/location/update	POST	Peer location update when online	JWT

Socket.IO events used in Online Trip mode:

Table A.2: Socket.IO Event Specifications

Event Name	Direction	Payload	Purpose
join_trip	Client → Server	{tripId, userId}	Join Online Trip room
new_message	Client → Server	MessageModel JSON	Send new text message
message_received	Server → Client	MessageModel JSON	Receive new message broadcast
member_online	Server → Client	{userId, displayName}	Peer online presence update
member_offline	Server → Client	{userId}	Peer disconnected from trip
sos_alert	Server → Client	SosEventModel JSON	Server-relayed SOS alert

Appendix B: SQLite Database Schema and Full Table Definitions

The complete SQLite schema is managed through Dart migration scripts using the sqflite package. Table creation queries:

```
CREATE TABLE messages (id TEXT PRIMARY KEY, trip_id TEXT NOT NULL, sender_id TEXT NOT NULL, sender_display_name TEXT, content TEXT NOT NULL, message_type TEXT DEFAULT "text", status TEXT DEFAULT "pending", priority TEXT DEFAULT "normal", ttl INTEGER DEFAULT 5, hop_count INTEGER DEFAULT 0, relay_uuid TEXT, local_only INTEGER DEFAULT 0, created_at TEXT NOT NULL, delivered_at TEXT, synced_at TEXT);

CREATE TABLE peer_metric_samples (id INTEGER PRIMARY KEY AUTOINCREMENT, endpoint_id TEXT NOT NULL, rssi REAL, ack_rtt_ms REAL, packet_success_rate REAL, retry_count INTEGER DEFAULT 0, disconnect_count INTEGER DEFAULT 0, status TEXT, created_at TEXT NOT NULL);

CREATE TABLE spatial_fixes (id INTEGER PRIMARY KEY AUTOINCREMENT, peer_id TEXT NOT NULL, display_name TEXT, latitude REAL NOT NULL, longitude REAL NOT NULL, accuracy REAL, bearing REAL, speed REAL, altitude REAL, timestamp TEXT NOT NULL);

CREATE TABLE sos_events (id TEXT PRIMARY KEY, sender_id TEXT NOT NULL, sender_display_name TEXT, latitude REAL, longitude REAL, message TEXT, battery_level INTEGER, status TEXT DEFAULT "broadcasting", created_at TEXT NOT NULL, acknowledged_at TEXT, synced_at TEXT);

CREATE TABLE deduplication_ledger (packet_uuid TEXT PRIMARY KEY, received_at TEXT NOT NULL);
```

Appendix C: Empirical Test Protocol and Traceability Matrix

All field trials followed a documented test protocol with the following steps for each experiment configuration:

1. Charge all test devices to >80% battery; record initial battery levels.
2. Install the same APK build (commit e82b185) on all test devices.
3. Clear the SQLite database on all devices using the built-in debug reset function.
4. Place devices in the specified physical configuration (1-hop, 2-hop, etc.).
5. Start ADB logging on the designated monitoring device.
6. Execute the test script: transmit 1,000 packets at 5-second intervals.
7. Record: delivered count, duplicate count, RTT per packet from ADB logs.
8. Calculate PDR, mean RTT, and max RTT from the log data.
9. Record final battery levels and ambient temperature.

10. Repeat steps 1–9 for each hop configuration and environment.

Appendix D: Academic Integrity and AI Use Disclosure

All four members of Research Group R26-IT-135 used generative AI writing assistance (specifically, large language model tools) to help structure, draft, and refine portions of this group thesis. The following disclosure applies collectively:

- Generative AI was used for drafting text, improving clarity, and structuring sections of the thesis. All AI-generated content was reviewed, verified, and edited by the respective student authors.
- No AI-generated content was included without human verification. All technical claims, experimental results, implementation details, and citations are based on actual project work and verified sources.
- No AI tool was used to generate experimental data, performance metrics, or implementation code. All code in the TrailLink repository was written by the student authors.
- All student authors remain fully responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis, in accordance with SLIIT academic integrity policy.